UI DESIGN-FLUTTER

B.Tech. III Year I Sem. L T P C
0 0 2 1

Course Objectives:

- Learns to Implement Flutter Widgets and Layouts
- Understands Responsive UI Design and with Navigation in Flutter
- Knowledge on Widgets and customize widgets for specific UI elements, Themes
- Understand to include animation apart from fetching data

Course Outcomes:

- Implements Flutter Widgets and Layouts
- Responsive UI Design and with Navigation in Flutter
- Create custom widgets for specific UI elements and also Apply styling using themes and custom styles.
- Design a form with various input fields, along with validation and error handling
- Fetches data and write code for unit Test for UI components and also animation

List of Experiments

Exp. No.	Experiment Name	Page No.

1	a) Install Flutter and Dart SDK b) Write a simple Dart program to understand the language basics.	
2	a) Explore various Flutter widgets (Text, Image, Container, etc.). b) Implement different layout structures using Row, Column, and Stack widgets	
3	a) Design a responsive UI that adapts to different screen sizes. b) Implement media queries and breakpoints for responsiveness.	
4	a) Set up navigation between different screens using Navigator. b) Implement navigation with named routes.	
5	a) Learn about stateful and stateless widgets. b) Implement state management using set State and Provider	
6	a) Create custom widgets for specific UI elements. b) Apply styling using themes and custom style	
7	a) Design a form with various input fields. b) Implement form validation and error handling.	
8	a) Add animations to UI elements using Flutter's animation framework. b) Experiment with different types of animations (fade, slide, etc.).	
9	a) Fetch data from a REST API. b) Display the fetched data in a meaningful way in the UI.	
10	a) Write unit tests for UI components. b) Use Flutter's debugging tools to identify and fix issues.	

UI design with Flutter typically provides a structured guide for students to learn and practice designing user interfaces using Flutter, a popular framework for building cross-platform mobile applications. The manual usually covers various topics, including:
Introduction to Flutter: An overview of Flutter, its features, and benefits for mobile app development.

Setting Up Development Environment: Instructions for setting up Flutter and its dependencies on different platforms like Windows, macOS, and Linux.

Flutter Basics: Fundamentals of Flutter, including widgets, layouts, navigation, state management, and basic UI design principles.

Building UI Components: Step-by-step tutorials for creating common UI components such as buttons, text fields, lists, and forms using Flutter widgets.

Styling and Theming: Guidelines for applying styles, themes, colors, fonts, and other visual elements to enhance the appearance of the app.

Responsive Design: Techniques for designing responsive user interfaces that adapt to different screen sizes and orientations.

Navigation and Routing: How to implement navigation between screens and manage application routes using Flutter's navigation system.

State Management: Introduction to state management techniques in Flutter, including setState, Provider, Bloc, and other state management libraries.

API Integration: Integration of RESTful APIs to fetch and display data in the app, including handling asynchronous operations and error handling.

Testing and Debugging: Strategies for testing UI components, debugging common issues, and using Flutter's debugging tools effectively.

Objectives of the laboratory sessions
The objectives of laboratory sessions in UI design with Flutter typically include:
Understanding Flutter Framework: Gain familiarity with the Flutter framework, its architecture, and key concepts for building cross-platform mobile applications.

Practical Experience: Provide hands-on experience to students in designing user interfaces using Flutter widgets, layouts, and components.

Applying UI Design Principles: Apply fundamental principles of UI design, such as consistency, clarity, and usability, to create intuitive and visually appealing user interfaces.

Implementing Responsive Design: Learn techniques for designing responsive user interfaces that adapt to different screen sizes and orientations, ensuring a consistent user experience across devices.

Styling and Theming: Understand how to apply styles, themes, colors, and typography to enhance the visual appearance of the app and maintain brand consistency.

Navigation and Routing: Learn how to implement navigation between screens and manage application routes using Flutter's navigation system.

State Management: Explore various state management techniques in Flutter, such as setState, Provider, Bloc, and others, to manage and update the state of UI components.

API Integration: Integrate RESTful APIs to fetch and display dynamic data in the app, including handling asynchronous operations and error handling.

Testing and Debugging: Practice testing UI components, debugging common issues, and using Flutter's debugging tools effectively to identify and fix errors.

Deployment and Publishing: Understand the process of building and deploying Flutter apps to different platforms like Android and iOS, and publishing them to app stores.

Collaboration and Communication: Foster collaboration and communication skills by working in teams on UI design projects, sharing ideas, and providing feedback to peers.

Problem-Solving Skills: Develop problem-solving skills by identifying UI design challenges, brainstorming solutions, and implementing effective design solutions using Flutter.

Explore Algorithmic Complexity and Efficiency: Analyze the time and space complexity of differentAI algorithms, and develop an understanding of algorithmic efficiency and optimization strategies.

Enhance Critical Thinking and Problem-Solving Abilities: Engage in critical thinking and analytical reasoning while designing and implementing solutions to complex AI problems, fostering problem-solving skills essential for AI development.

Experiment with Different Approaches: Experiment with various AI approaches, including search algorithms, heuristic methods, machine learning techniques, and optimization algorithms, to understandtheir strengths, weaknesses, and applications.

Foster Collaboration and Communication: Collaborate with peers on problem-solving tasks, discuss algorithm design choices, and communicate findings effectively, promoting teamwork and effective communication skills.

Build a Strong Foundation in AI Theory and Practice: Develop a strong foundation in both theoretical concepts and practical applications of artificial intelligence, preparing students for furtherstudy or professional work in the field.

Encourage Creativity and Innovation: Encourage creativity and innovation in problem-solving, allowing students to explore alternative solutions, adapt existing algorithms, and develop novel approaches to AI challenges.

Promote Lifelong Learning in AI: Cultivate a passion for lifelong learning and exploration in the field of artificial intelligence, inspiring students to continue their education and pursue advancements inAI technology throughout their careers.

UI design with Flutter, students should ideally have thefollowing prerequisites:

Basic Programming Skills: Students should have a basic understanding of programming concepts, including variables, data types, control structures, and functions. Knowledge of object-oriented programming (OOP) concepts like classes, objects, and inheritance would be beneficial.

Familiarity with Dart Programming Language: Flutter uses the Dart programming language, so students should have a basic understanding of Dart syntax, features, and concepts. Topics such as variables, functions, classes, and asynchronous programming with Future and Stream would be important.

Understanding of UI Design Principles: Familiarity with fundamental principles of user interface (UI) design, such as consistency, simplicity, visual hierarchy, and usability, would be helpful. Students should have a basic understanding of how to design user-friendly and visually appealing interfaces.

Experience with Mobile Development: While not mandatory, prior experience with mobile app development using frameworks like Flutter, React Native, or native development (Android/iOS) wouldbe advantageous. This includes understanding app lifecycle, navigation patterns, and platform-specific

considerations.

Development Environment Setup: Students should be able to set up and configure their developmentenvironment for Flutter development. This includes installing Flutter SDK, Dart SDK, and necessary development tools like IDE (e.g., Android Studio, IntelliJ IDEA, Visual Studio Code) and Flutter plugins/extensions.

Problem-Solving Skills: Students should have strong problem-solving skills and the ability to troubleshoot issues independently. This includes debugging code, researching solutions online, andseeking help when needed.

Critical Thinking and Creativity: UI design involves creative thinking and problem-solving to create intuitive and aesthetically pleasing user interfaces. Students should be able to think critically, analyze design requirements, and propose innovative solutions.

Time Management: Managing time effectively is crucial for completing lab assignments, meeting deadlines, and optimizing productivity during lab sessions. Students should be able to prioritize tasks, allocate time wisely, and stay organized.

Effective Communication: Communication skills, both written and verbal, are essential for collaborating with peers, discussing design ideas, and presenting project outcomes. Students should be able to articulate their thoughts, provide constructive feedback, and work effectively in teams.

While these prerequisites provide a foundation for successful participation in UI design laboratory sessions, students with varying levels of experience and backgrounds can benefit from engaging with the course material and actively participating in hands-on activities and projects.

1.1 Necessary software and hardware requirements

UI design laboratory sessions with Flutter, students will need the followingsoftware and hardware requirements:

Software Requirements:

Flutter SDK: Install the Flutter SDK, which includes the Flutter framework and the Dart programming language. Flutter provides installation instructions for various platforms such as Windows, macOS, and Linux.

Integrated Development Environment (IDE):

Recommended: Android Studio with Flutter plugin or Visual Studio Code with Flutter extension. Both IDEs provide excellent support for Flutter development and include features

like code completion, debugging, and project management tools.

Alternatively, students can use IntelliJ IDEA with Flutter plugin or any other text editor of their choice, but full-featured IDEs are recommended for better productivity.

Dart SDK: The Dart SDK is included with the Flutter SDK, but students can also install it separately ifneeded. It provides command-line tools for Dart development and is required for

running Dart programs outside of Flutter projects.

Mobile Emulators or Devices:

Install Android Studio and set up Android Virtual Device (AVD) for running Android emulators.

For iOS development, students will need a Mac computer with Xcode installed to run the iOS simulator or test apps on physical iOS devices.

Package Management: Flutter uses Dart's package manager, Pub, to manage dependencies. Students should have access to the internet to download packages from Pub and add them to their Flutter projects.

Hardware Requirements:

Computer: Students will need a computer (desktop or laptop) running Windows, macOS, or Linux to develop Flutter applications. The computer should meet the minimum system requirements for runningthe chosen IDE and emulators/simulators.

Mobile Devices (Optional): While emulators/simulators are sufficient for testing and development, students may also want to test their apps on physical Android and iOS devices. Ensure compatibility with Flutter by enabling developer mode and USB debugging on the devices.

Internet Connection: A stable internet connection is necessary for downloading Flutter SDK, IDEs, packages from Pub, and accessing online resources and documentation.

Storage: Sufficient storage space on the computer to install Flutter SDK, IDEs, and other development tools, as well as to store project files and dependencies.

By meeting these software and hardware requirements, students can effectively engage in UI design laboratory sessions with Flutter, develop mobile applications, and gain practical experience in building user interfaces for various platforms.

Minimum System requirements:

The minimum system requirements for developing Flutter applications are relatively modest, but theymay vary slightly depending on the operating system and the specific development tools being used. Here are the general minimum system requirements:

For Windows:

Operating System: Windows 7 SP1 or later (64-bit)**Processor**: Intel Core i3 or AMD equivalent (64-bit)**Memory (RAM)**: 4GB RAM or more

Storage: 2GB of free disk space

Graphics Card: Integrated graphics or dedicated GPU with DirectX 9 support

Internet Connection: Required for downloading development tools, SDKs, and dependencies

For macOS:

Operating System: macOS 10.12 (Sierra) or later

Processor: Intel Core i3 or higher **Memory (RAM)**: 4GB RAM or more**Storage**: 2GB of free disk space

Graphics Card: Integrated graphics or dedicated GPU

Internet Connection: Required for downloading development tools, SDKs, and dependencies

For Linux:

Operating System: Ubuntu 16.04 LTS (Xenial Xerus) or later, or another Linux distribution withsimilar support

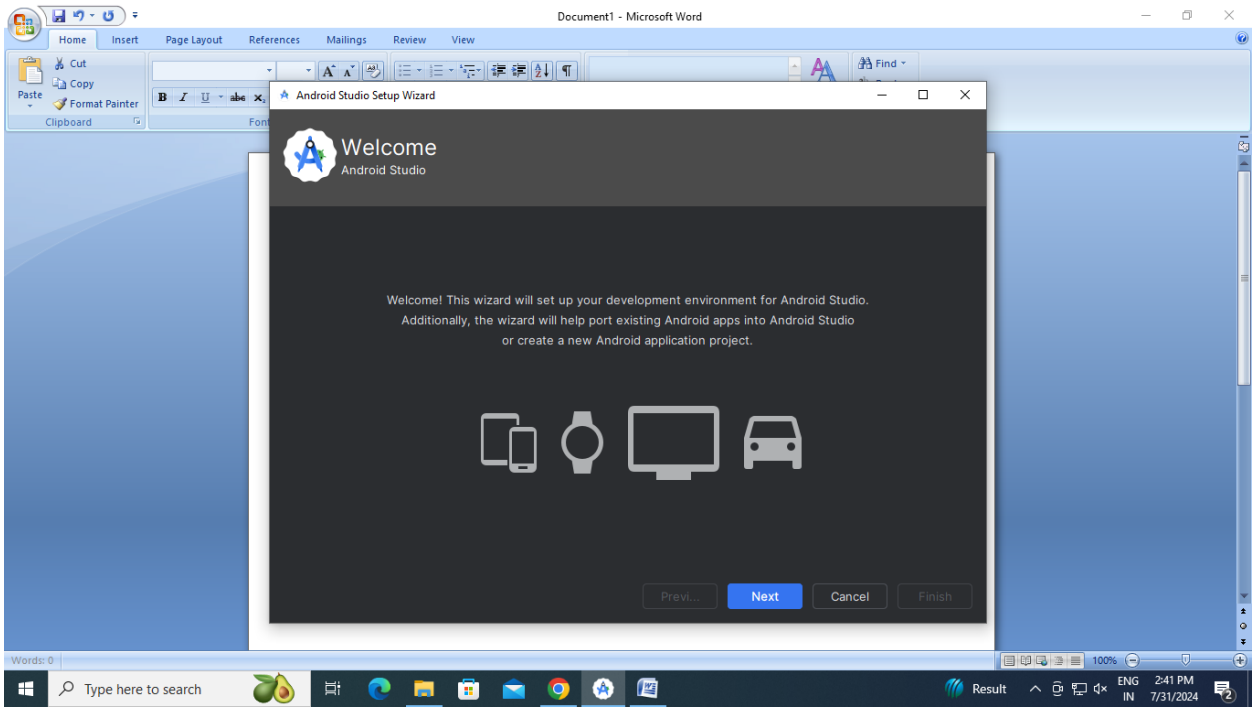
Processor: Intel Core i3 or AMD equivalent**Memory (RAM)**: 4GB RAM or more
Storage: 2GB of free disk space

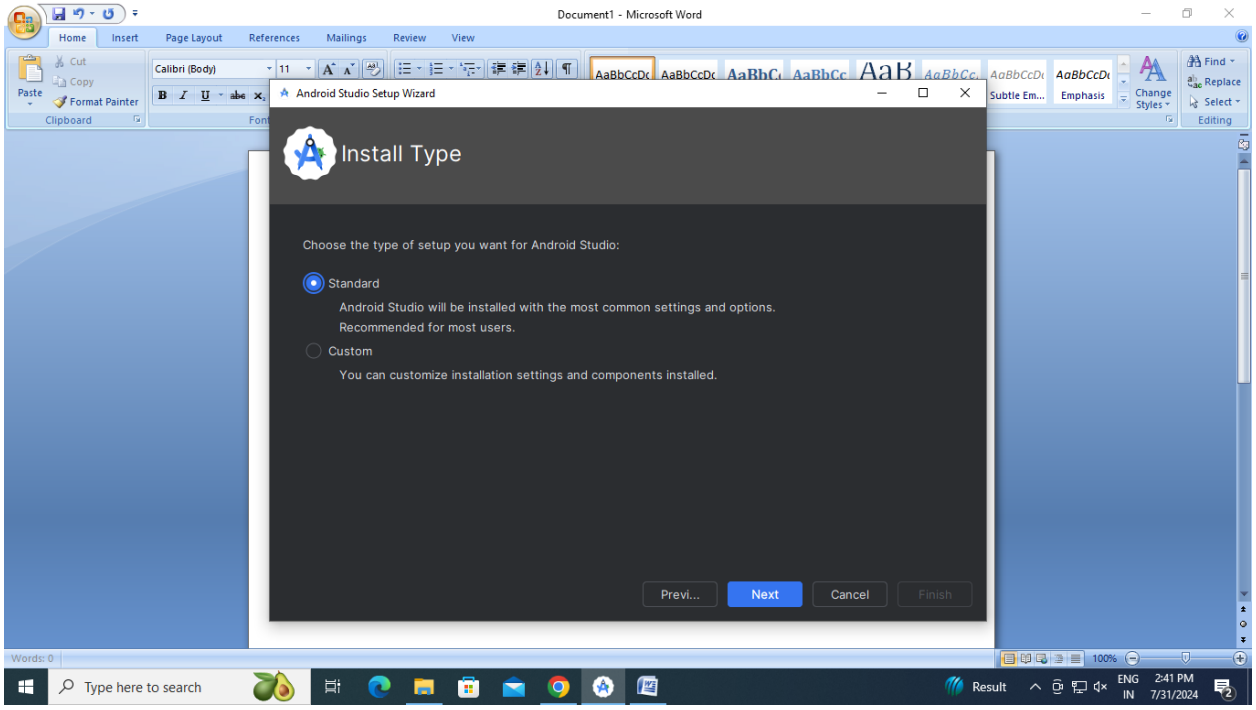
Graphics Card: Integrated graphics or dedicated GPU

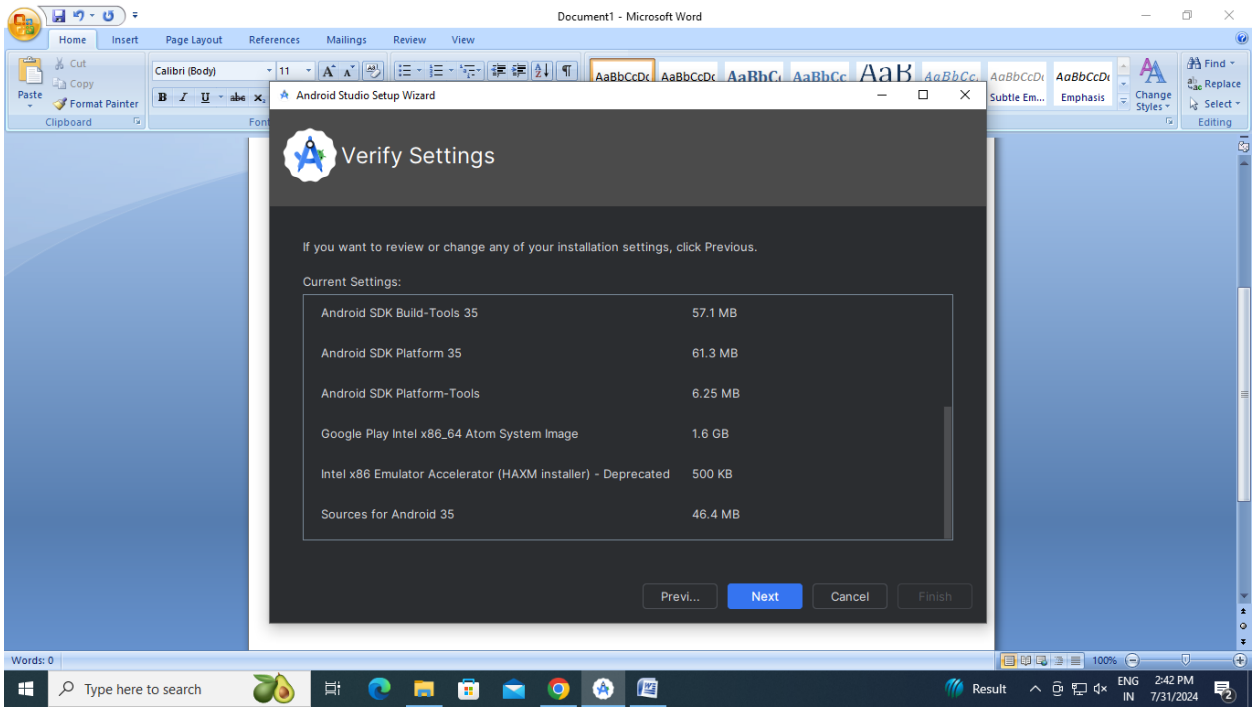
Internet Connection: Required for downloading development tools, SDKs, and dependencies

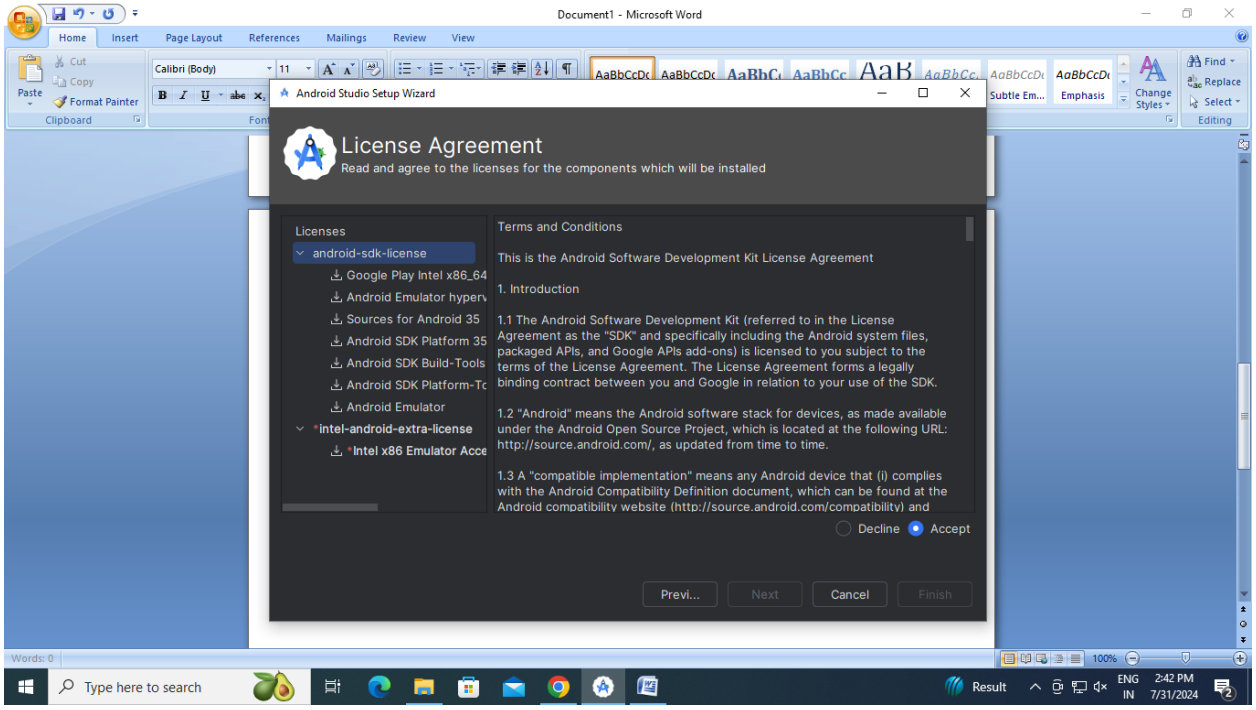
These are general guidelines, and the actual system requirements may vary depending on factors such asthe size and complexity of the Flutter projects, the number of plugins and dependencies used, and the performance of the development tools (IDEs) being used. Additionally, running emulators or simulatorsfor testing on mobile devices may require more system resources.

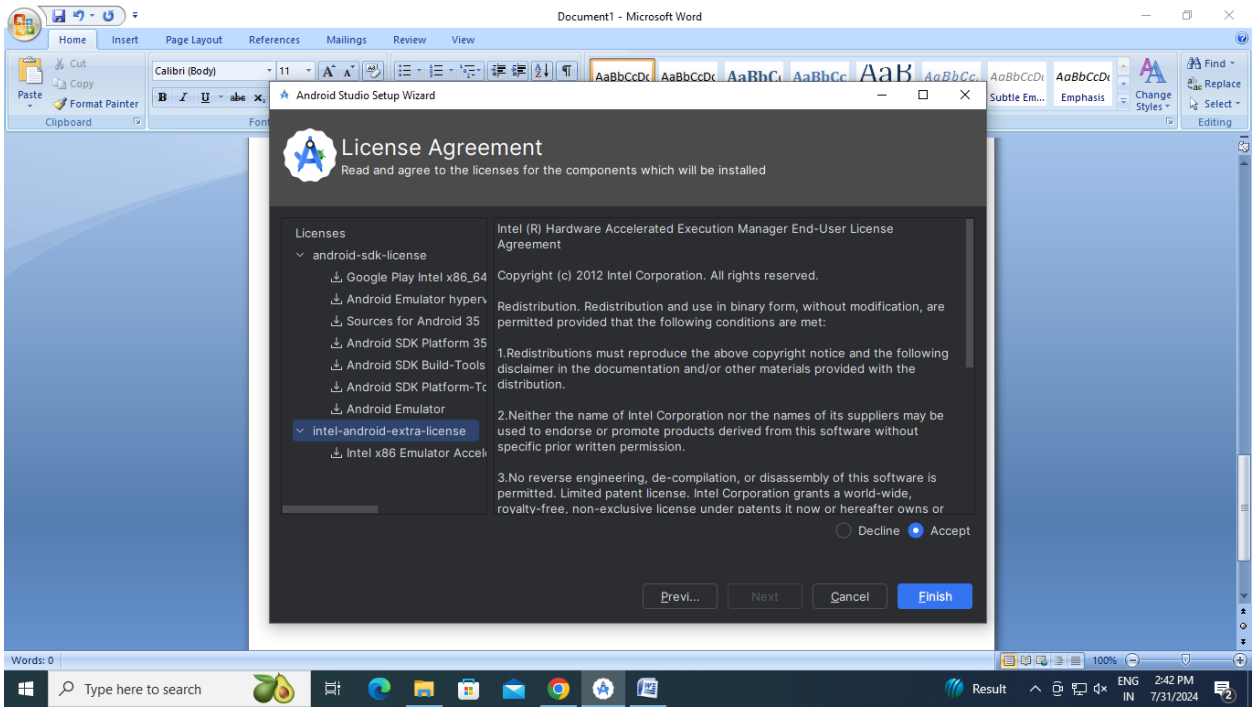
1 a) Install Flutter and Dart SDK

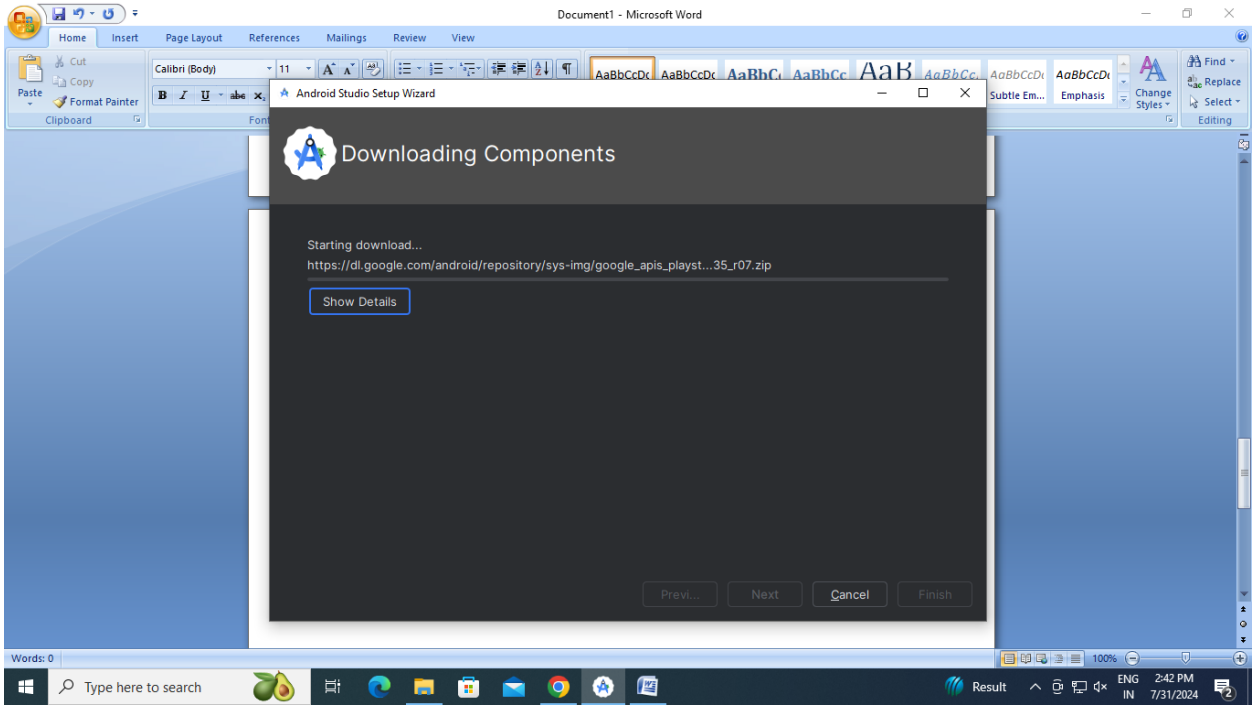

Document1 - Microsoft Word
Home
Insert
Page Layout
References
Mailings
Review
View
Paste
Cut
Copy
Format Painter
Clipboard
Find
Android Studio Setup Wizard
Welcome
Android Studio
Welcome! This wizard will set up your development environment for Android Studio.
Additionally, the wizard will help port existing Android apps into Android Studio
or create a new Android application project.
Previ...
Next
Cancel
Finish
Words: 0
100%
Type here to search
Result
ENG
IN
2:41 PM
7/31/2024

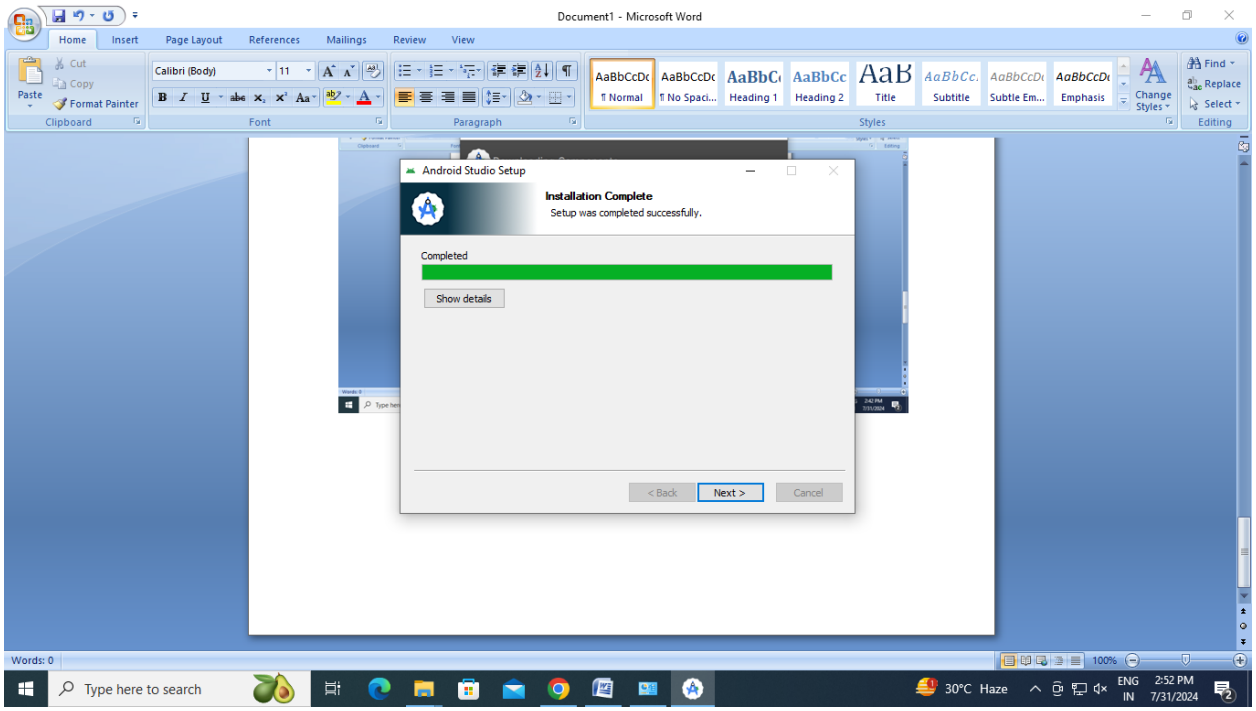

Document1 - Microsoft Word
Home
Insert
Page Layout
References
Mailings
Review
View
Paste
Cut
Copy
Format Painter
Clipboard
Calibri (Body)
11
Subtle Em...
Emphasis
Change Styles
Find
Replace
Select
Editing
Android Studio Setup Wizard
Install Type
Choose the type of setup you want for Android Studio:
Standard
Android Studio will be installed with the most common settings and options.
Recommended for most users.
Custom
You can customize installation settings and components installed.
Previ...
Next
Cancel
Finish
Words: 0
100%
Type here to search
Result
ENG
IN
2:41 PM
7/31/2024

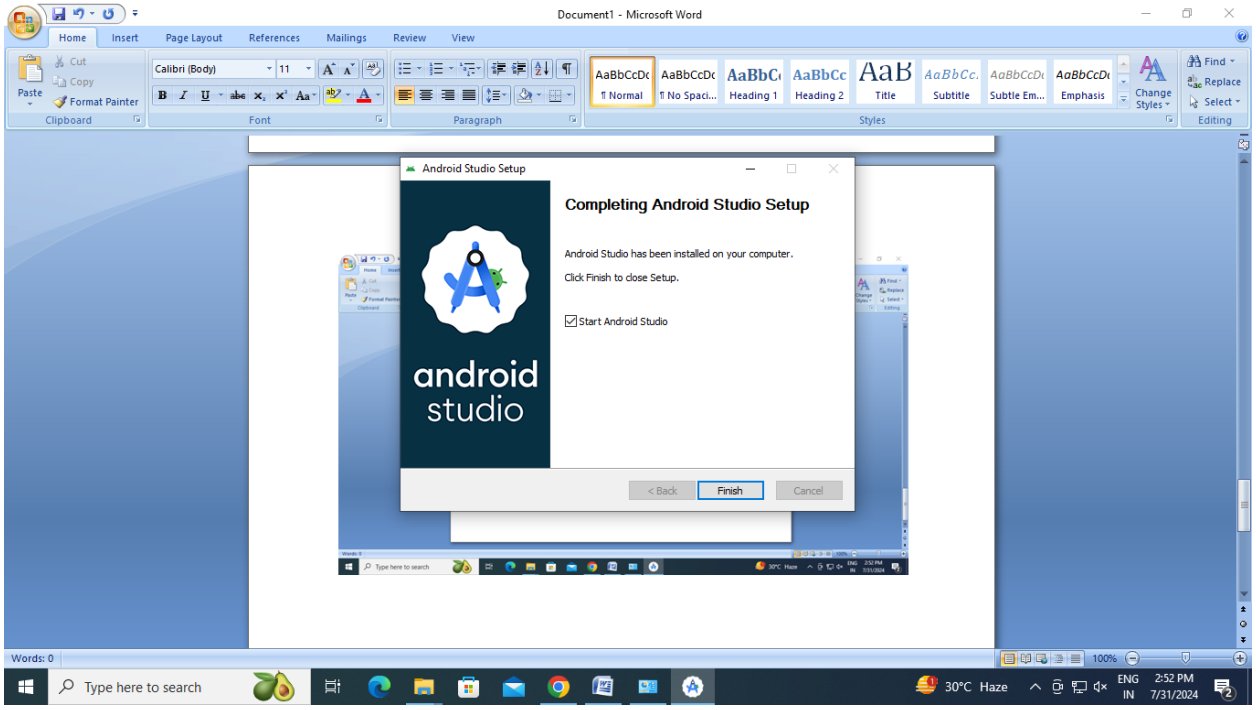

Document1 - Microsoft Word
Android Studio Setup Wizard
Verify Settings
If you want to review or change any of your installation settings, click Previous.
Current Settings:
Android SDK Build-Tools 35 57.1 MB
Android SDK Platform 35 61.3 MB
Android SDK Platform-Tools 6.25 MB
Google Play Intel x86_64 Atom System Image 1.6 GB
Intel x86 Emulator Accelerator (HAXM installer) - Deprecated 500 KB
Sources for Android 35 46.4 MB
Previ...
Next
Cancel
Finish

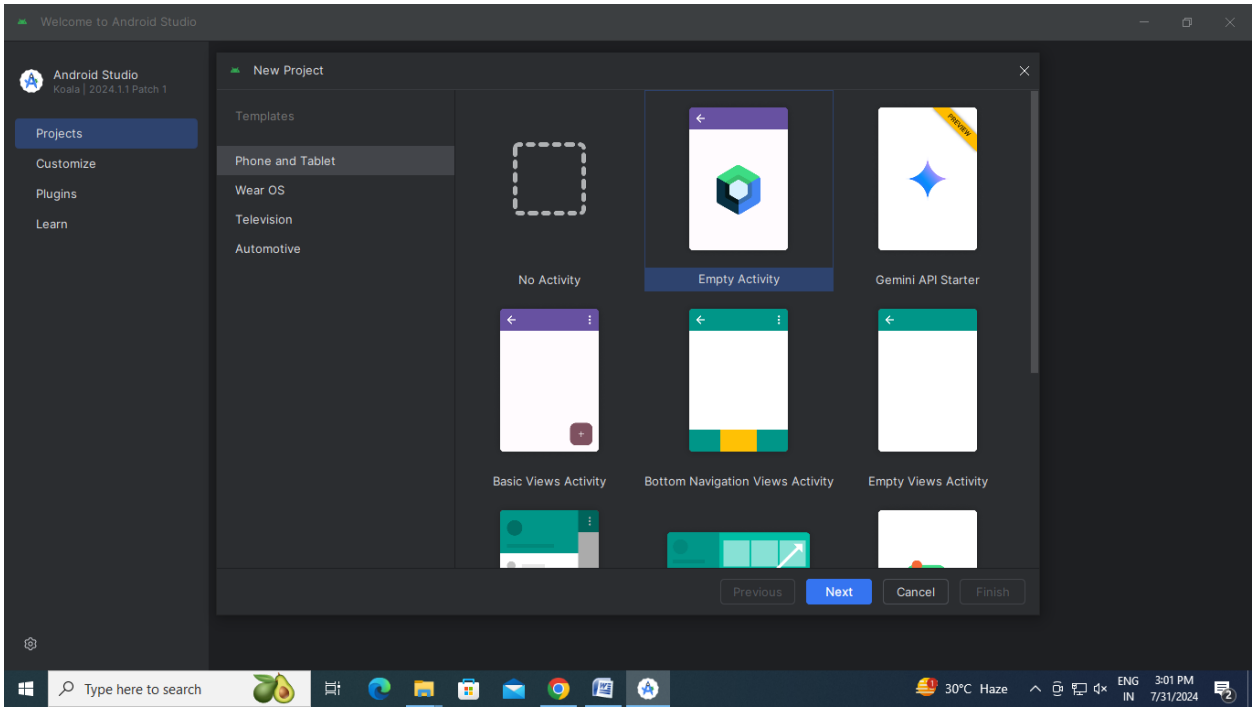

Document1 - Microsoft Word
Android Studio Setup Wizard
License Agreement
Read and agree to the licenses for the components which will be installed
Licenses
android-sdk-license
Google Play Intel x86_64
Android Emulator hyperv
Sources for Android 35
Android SDK Platform 35
Android SDK Build-Tools
Android SDK Platform-Tc
Android Emulator
*intel-android-extra-license
*Intel x86 Emulator Acce
Terms and Conditions
This is the Android Software Development Kit License Agreement
1. Introduction
1.1 The Android Software Development Kit (referred to in the License Agreement as the "SDK" and specifically including the Android system files, packaged APIs, and Google APIs add-ons) is licensed to you subject to the terms of the License Agreement. The License Agreement forms a legally binding contract between you and Google in relation to your use of the SDK.
1.2 "Android" means the Android software stack for devices, as made available under the Android Open Source Project, which is located at the following URL: http://source.android.com/, as updated from time to time.
1.3 A "compatible implementation" means any Android device that (i) complies with the Android Compatibility Definition document, which can be found at the Android compatibility website (http://source.android.com/compatibility) and
Decline
Accept
Previ...
Next
Cancel
Finish

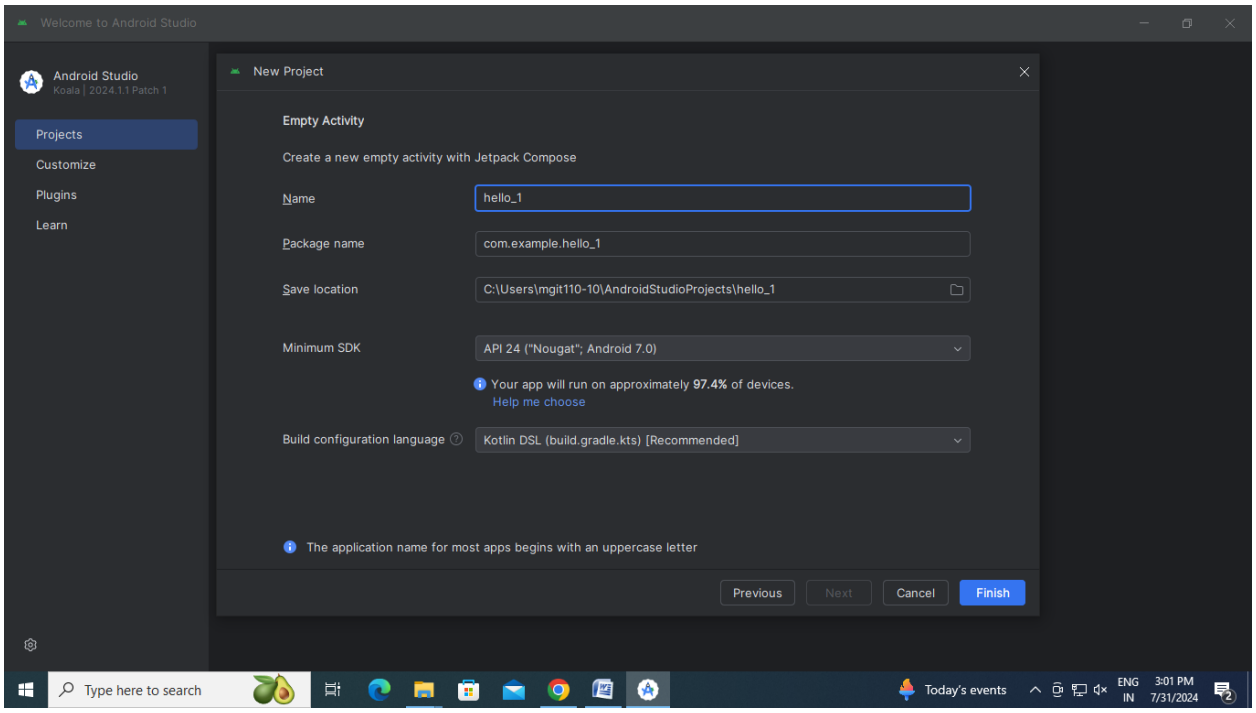

Document1 - Microsoft Word
Android Studio Setup Wizard
License Agreement
Read and agree to the licenses for the components which will be installed
Licenses
android-sdk-license
Google Play Intel x86_64
Android Emulator hyperv
Sources for Android 35
Android SDK Platform 35
Android SDK Build-Tools
Android SDK Platform-Tc
Android Emulator
intel-android-extra-license
Intel x86 Emulator Accel
Intel (R) Hardware Accelerated Execution Manager End-User License Agreement
Copyright (c) 2012 Intel Corporation. All rights reserved.
Redistribution. Redistribution and use in binary form, without modification, are permitted provided that the following conditions are met:
1.Redistributions must reproduce the above copyright notice and the following disclaimer in the documentation and/or other materials provided with the distribution.
2.Neither the name of Intel Corporation nor the names of its suppliers may be used to endorse or promote products derived from this software without specific prior written permission.
3.No reverse engineering, de-compilation, or disassembly of this software is permitted. Limited patent license. Intel Corporation grants a world-wide, royalty-free, non-exclusive license under patents it now or hereafter owns or
Decline
Accept
Previ...
Next
Cancel
Finish
Words: 0
Type here to search
Result
ENG
IN
2:42 PM
7/31/2024

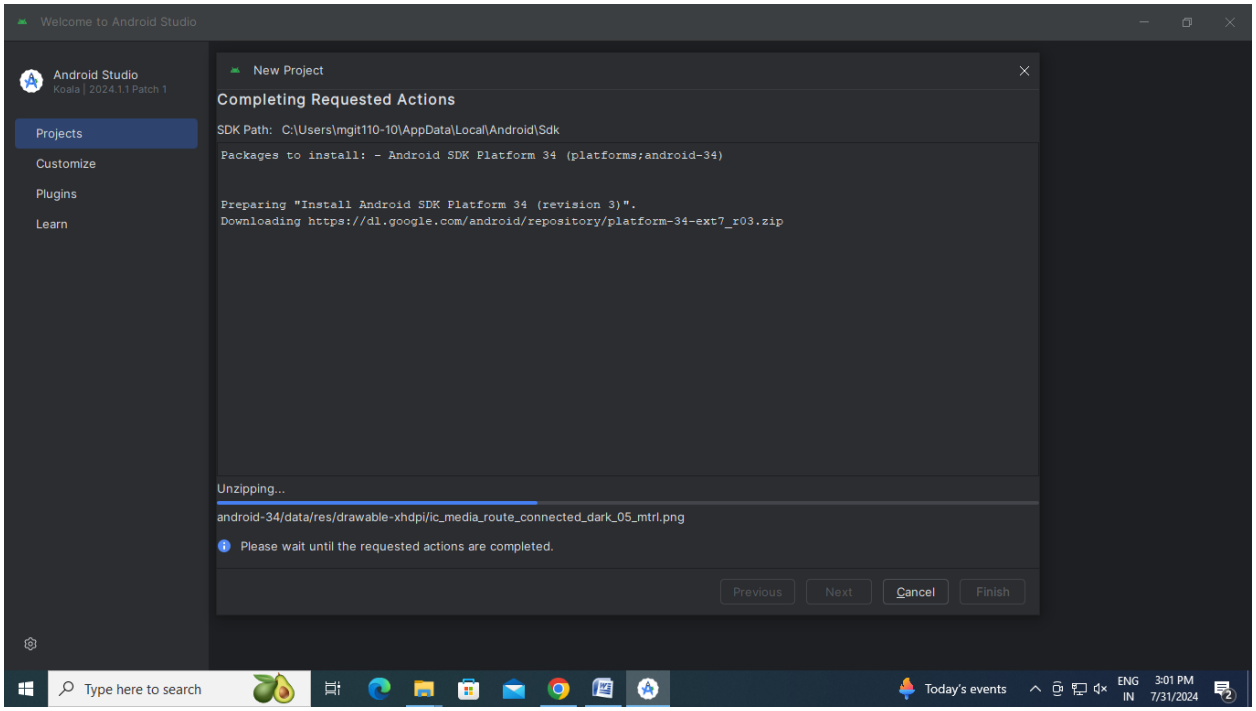

Document1 - Microsoft Word
Android Studio Setup Wizard
Downloading Components
Starting download...
https://dl.google.com/android/repository/sys-img/google_apis_playst...35_r07.zip
Show Details
Previ...
Next
Cancel
Finish
Words: 0
Type here to search
Result
ENG
IN
2:42 PM
7/31/2024

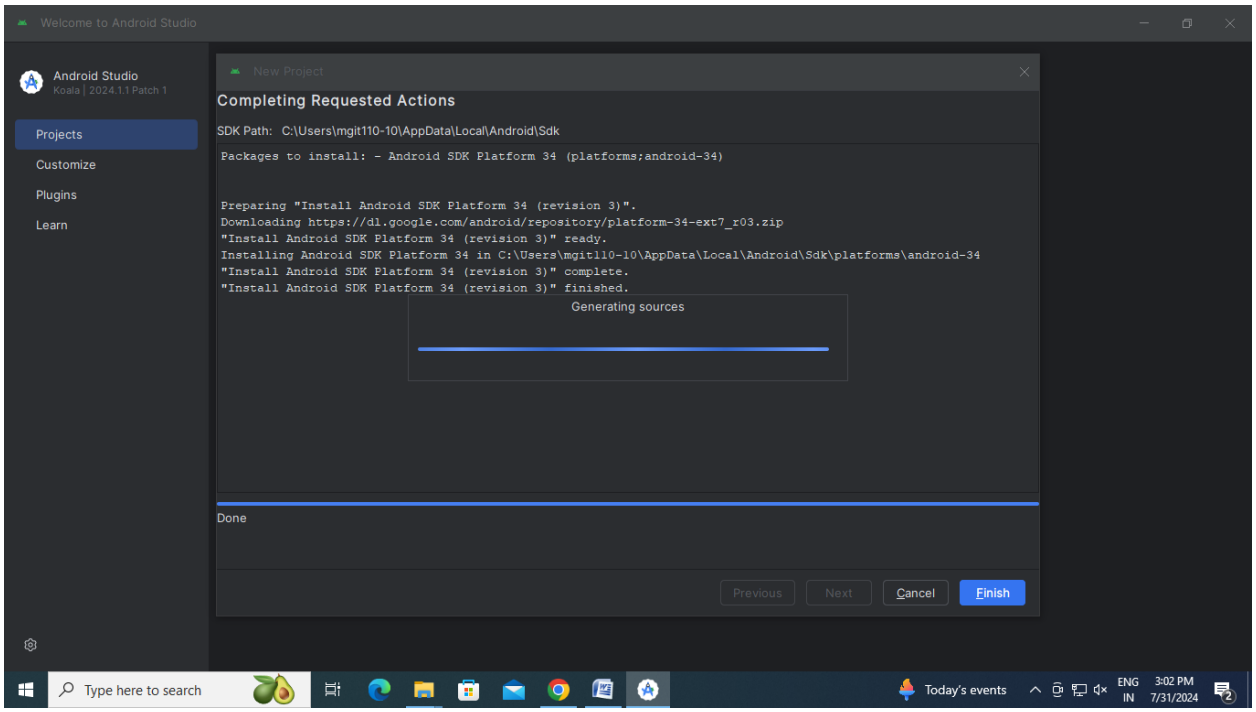

Document1 - Microsoft Word
Android Studio Setup
Installation Complete
Setup was completed successfully.
Completed
Show details
< Back
Next >
Cancel
Type here to search
30°C Haze
ENG
IN
2:52 PM
7/31/2024

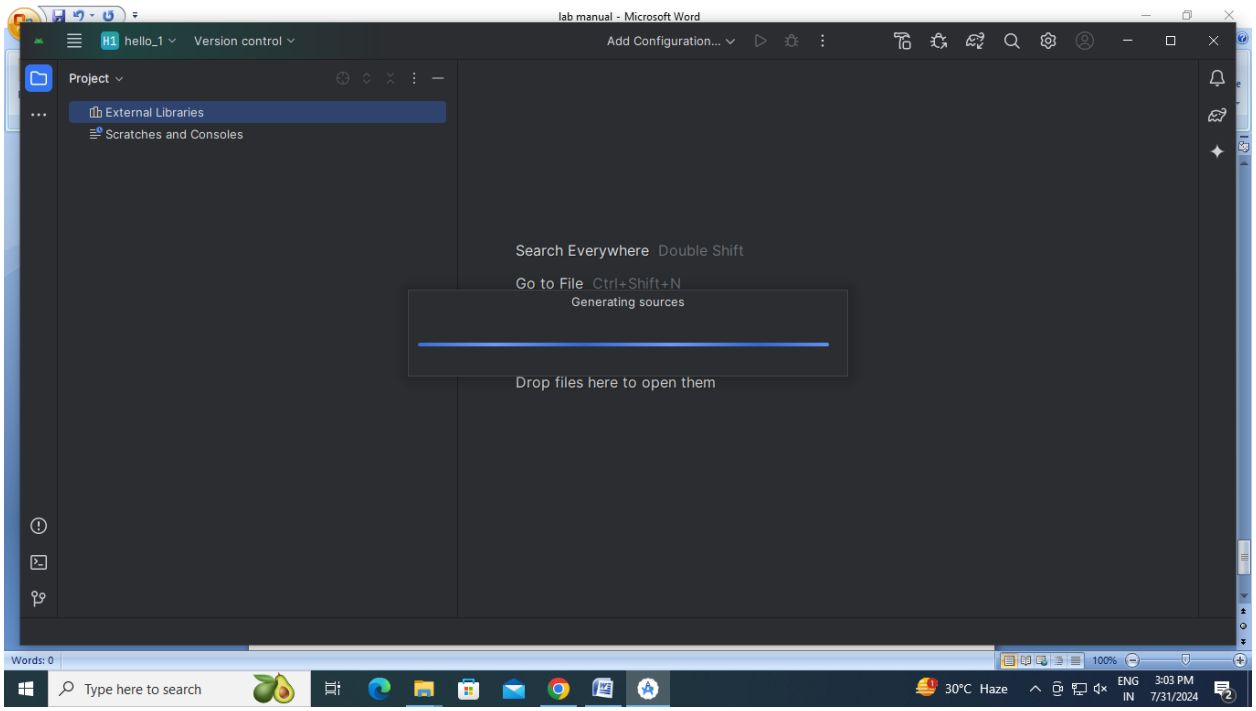

Document1 - Microsoft Word
Android Studio Setup
Completing Android Studio Setup
Android Studio has been installed on your computer.
Click Finish to close Setup.
Start Android Studio
android studio
< Back
Finish
Cancel
Type here to search
30°C Haze
ENG
IN
2:52 PM
7/31/2024

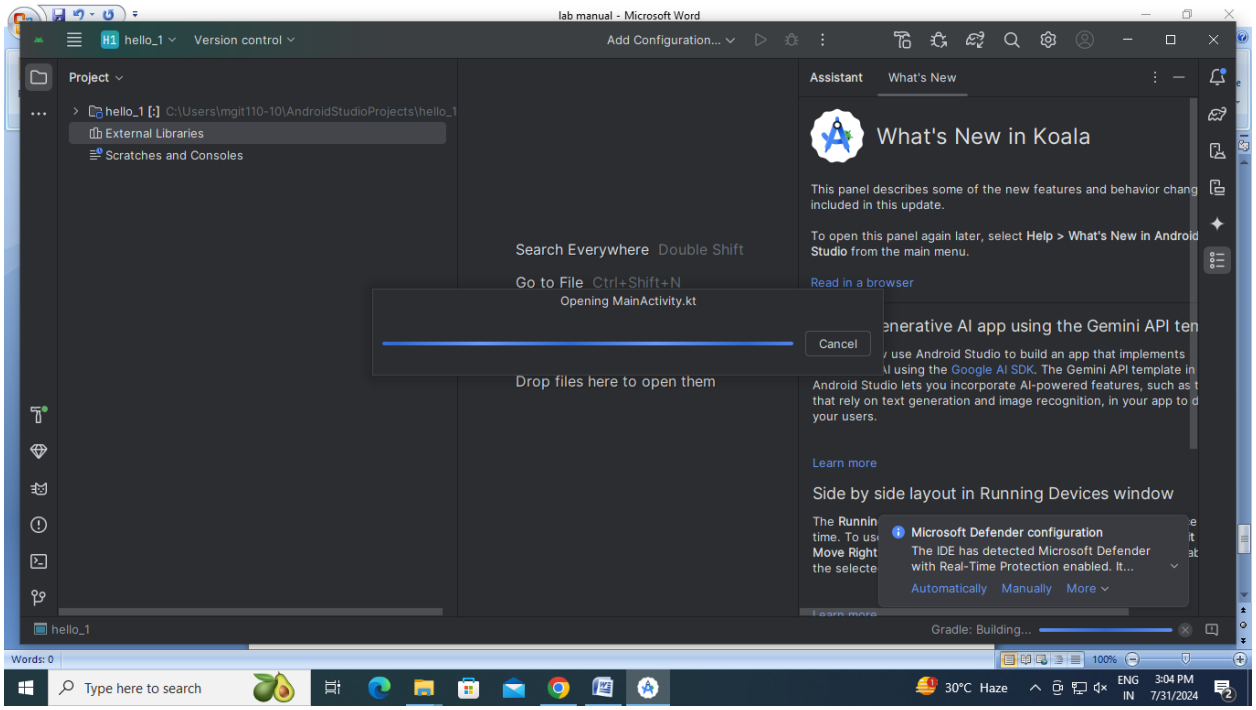

Welcome to Android Studio
Android Studio
Koala | 2024.1.1 Patch 1
Projects
Customize
Plugins
Learn
New Project
Templates
Phone and Tablet
Wear OS
Television
Automotive
No Activity
Empty Activity
Gemini API Starter
Basic Views Activity
Bottom Navigation Views Activity
Empty Views Activity
Previous
Next
Cancel
Finish
Type here to search
30°C Haze
ENG
IN
3:01 PM
7/31/2024

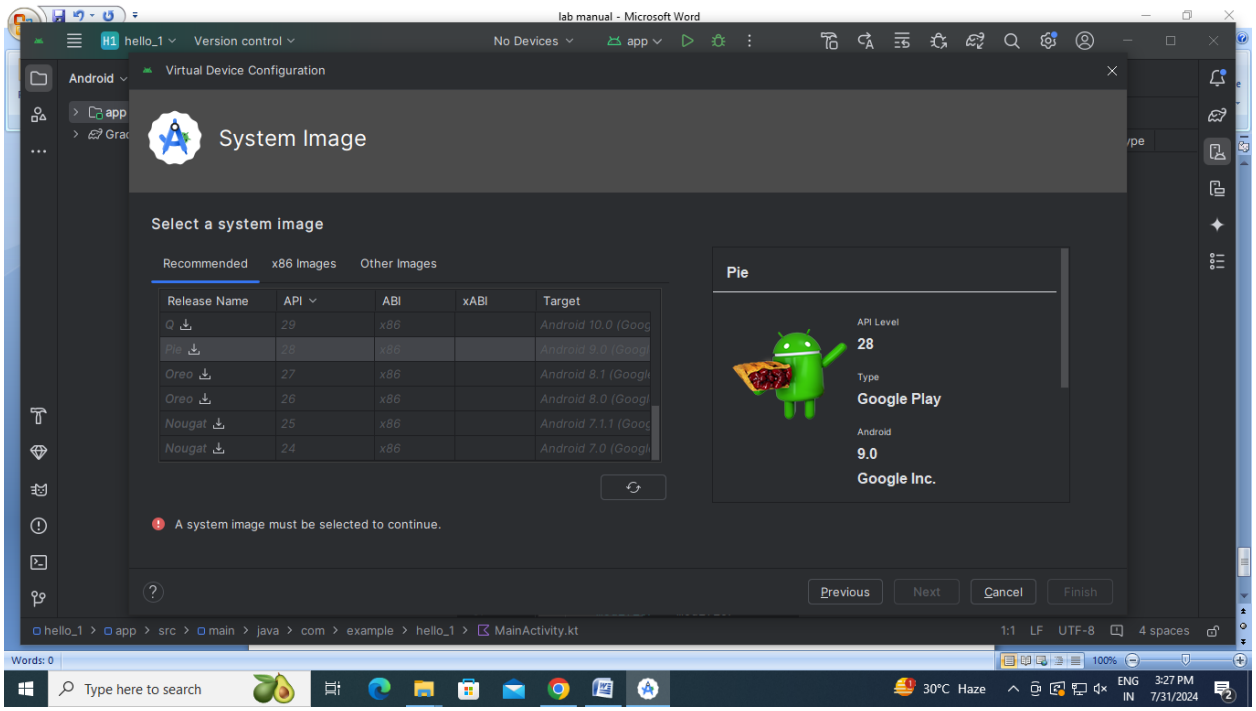

Welcome to Android Studio
Android Studio
Koala | 2024.1.1 Patch 1
Projects
Customize
Plugins
Learn
New Project
Empty Activity
Create a new empty activity with Jetpack Compose
Name
hello_1
Package name
com.example.hello_1
Save location
C:\Users\mgit110-10\AndroidStudioProjects\hello_1
Minimum SDK
API 24 ("Nougat"; Android 7.0)
Your app will run on approximately 97.4% of devices.
Help me choose
Build configuration language
Kotlin DSL (build.gradle.kts) [Recommended]
The application name for most apps begins with an uppercase letter
Previous
Next
Cancel
Finish
Type here to search
Today's events
ENG
IN
3:01 PM
7/31/2024

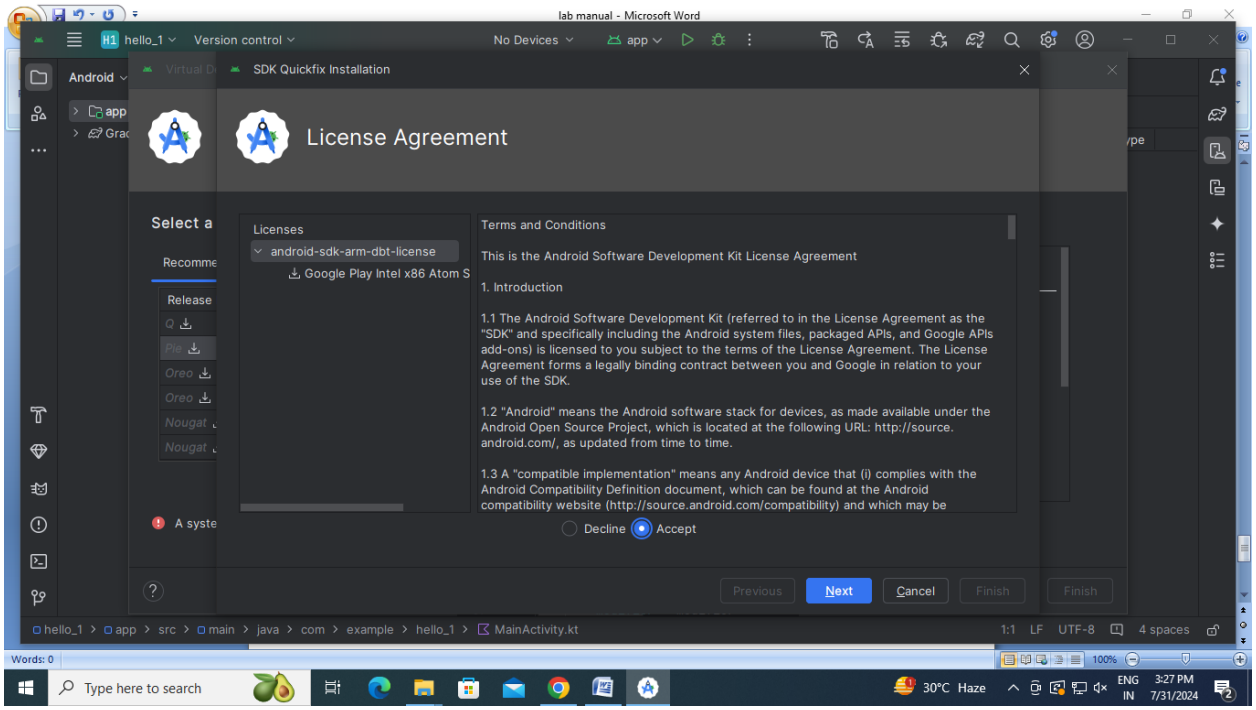

Welcome to Android Studio
Android Studio
Koala | 2024.1.1 Patch 1
Projects
Customize
Plugins
Learn
New Project
Completing Requested Actions
SDK Path: C:\Users\mgit110-10\AppData\Local\Android\Sdk
Packages to install: - Android SDK Platform 34 (platforms;android-34)
Preparing "Install Android SDK Platform 34 (revision 3)".
Downloading https://dl.google.com/android/repository/platform-34-ext7_r03.zip
Unzipping...
android-34/data/res/drawable-xhdpi/ic_media_route_connected_dark_05_mtrl.png
Please wait until the requested actions are completed.
Previous
Next
Cancel
Finish
Type here to search
Today's events
ENG
IN
3:01 PM
7/31/2024

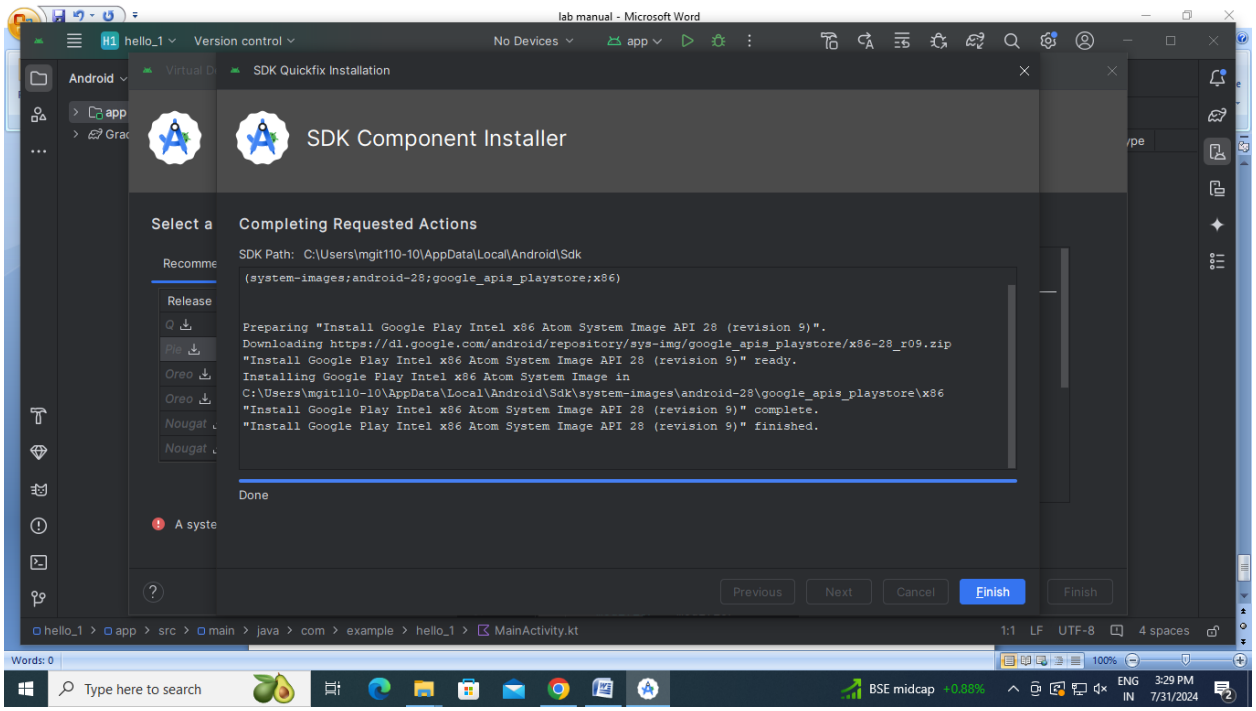

Welcome to Android Studio
Android Studio
Koala | 2024.1.1 Patch 1
Projects
Customize
Plugins
Learn
New Project
Completing Requested Actions
SDK Path: C:\Users\mgit110-10\AppData\Local\Android\Sdk
Packages to install: - Android SDK Platform 34 (platforms;android-34)
Preparing "Install Android SDK Platform 34 (revision 3)".
Downloading https://dl.google.com/android/repository/platform-34-ext7_r03.zip
"Install Android SDK Platform 34 (revision 3)" ready.
Installing Android SDK Platform 34 in C:\Users\mgit110-10\AppData\Local\Android\Sdk\platforms\android-34
"Install Android SDK Platform 34 (revision 3)" complete.
"Install Android SDK Platform 34 (revision 3)" finished.
Generating sources
Done
Previous
Next
Cancel
Finish
Type here to search
Today's events
ENG
IN
3:02 PM
7/31/2024

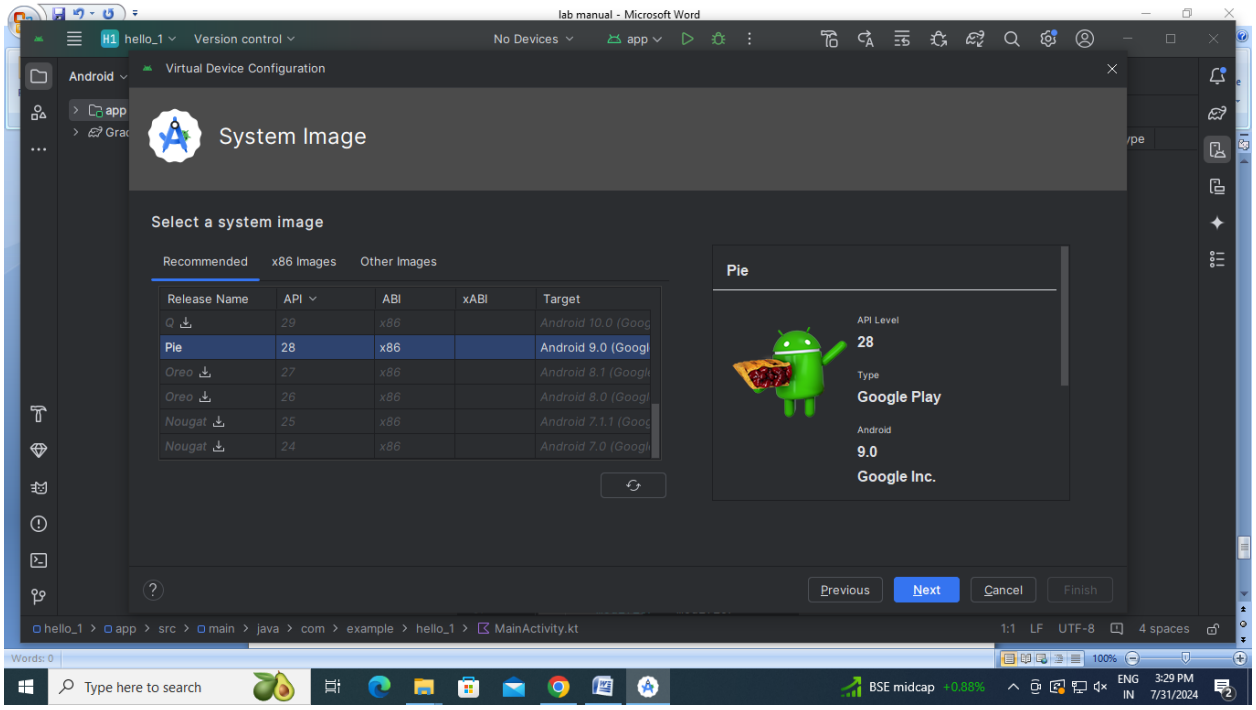

lab manual - Microsoft Word
hello_1
Version control
Add Configuration...
Project
External Libraries
Scratches and Consoles
Search Everywhere Double Shift
Go to File Ctrl+Shift+N
Generating sources
Drop files here to open them
Words: 0
100%
Type here to search
30°C Haze
ENG
IN
3:03 PM
7/31/2024

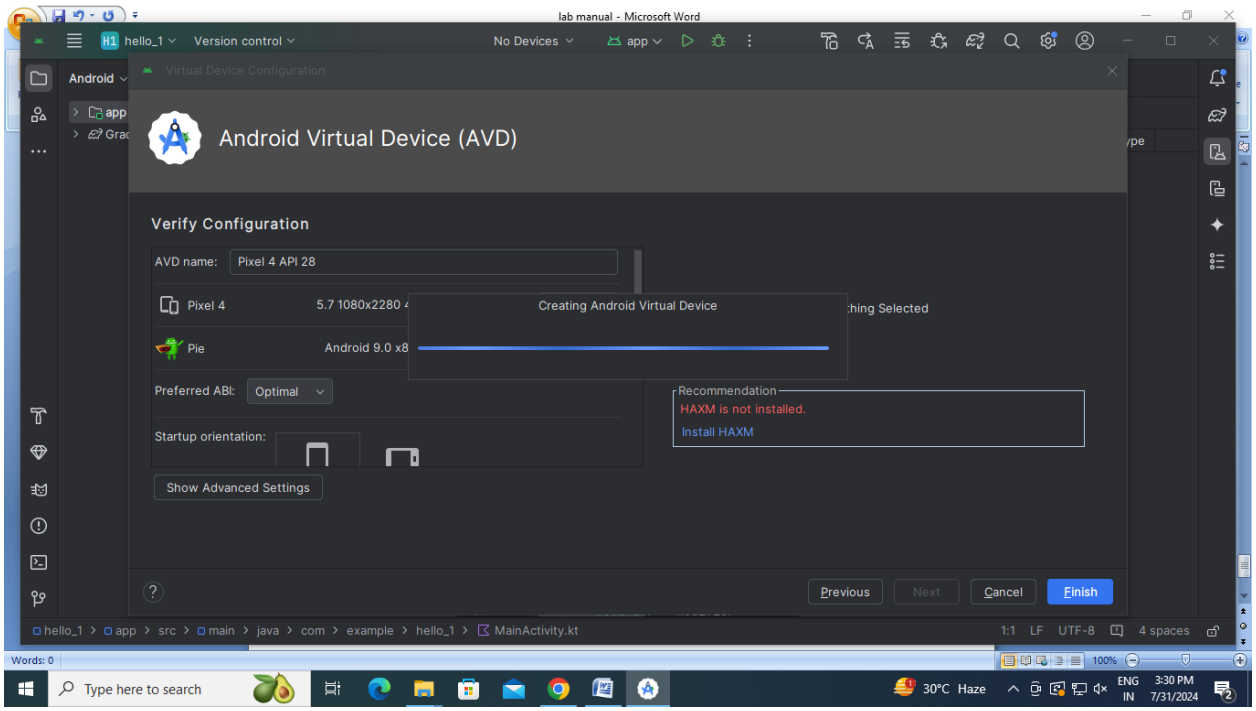

lab manual - Microsoft Word
hello_1
Version control
Add Configuration...
Project
hello_1 [:] C:\Users\mgit110-10\AndroidStudioProjects\hello_1
External Libraries
Scratches and Consoles
Assistant
What's New
What's New in Koala
This panel describes some of the new features and behavior chang
included in this update.
To open this panel again later, select Help > What's New in Android
Studio from the main menu.
Read in a browser
Search Everywhere Double Shift
Go to File Ctrl+Shift+N
Opening MainActivity.kt
Cancel
Drop files here to open them
Learn more
Side by side layout in Running Devices window
Microsoft Defender configuration
The IDE has detected Microsoft Defender
with Real-Time Protection enabled. It...
Automatically
Manually
More
hello_1
Gradle: Building...
Words: 0
100%
Type here to search
30°C Haze
ENG
IN
3:04 PM
7/31/2024


lab manual - Microsoft Word
hello_1
Version control
No Devices
app
Android
Virtual Device Configuration
System Image
Select a system image
Recommended
x86 Images
Other Images
Release Name
API
ABI
xABI
Target
Pie
API Level
28
Type
Google Play
Android
9.0
Google Inc.
A system image must be selected to continue.
Previous
Next
Cancel
Finish
hello_1 > app > src > main > java > com > example > hello_1 > MainActivity.kt
1:1
LF
UTF-8
4 spaces
Words: 0
100%
Type here to search
30°C Haze
ENG
IN
3:27 PM
7/31/2024


lab manual - Microsoft Word
hello_1
Version control
No Devices
app
Android
SDK Quickfix Installation
License Agreement
Licenses
android-sdk-arm-dbt-license
Google Play Intel x86 Atom S
Terms and Conditions
This is the Android Software Development Kit License Agreement
1. Introduction
1.1 The Android Software Development Kit (referred to in the License Agreement as the "SDK" and specifically including the Android system files, packaged APIs, and Google APIs add-ons) is licensed to you subject to the terms of the License Agreement. The License Agreement forms a legally binding contract between you and Google in relation to your use of the SDK.
1.2 "Android" means the Android software stack for devices, as made available under the Android Open Source Project, which is located at the following URL: http://source.android.com/, as updated from time to time.
1.3 A "compatible implementation" means any Android device that (i) complies with the Android Compatibility Definition document, which can be found at the Android compatibility website (http://source.android.com/compatibility) and which may be
Decline
Accept
Previous
Next
Cancel
Finish
hello_1 > app > src > main > java > com > example > hello_1 > MainActivity.kt
1:1
LF
UTF-8
4 spaces
Words: 0
100%
Type here to search
30°C Haze
ENG
IN
3:27 PM
7/31/2024


lab manual - Microsoft Word
hello_1
Version control
No Devices
app
SDK Quickfix Installation
SDK Component Installer
Select a
Completing Requested Actions
SDK Path: C:\Users\mgit110-10\AppData\Local\Android\Sdk
(system-images;android-28;google_apis_playstore;x86)
Preparing "Install Google Play Intel x86 Atom System Image API 28 (revision 9)".
Downloading https://dl.google.com/android/repository/sys-img/google_apis_playstore/x86-28_r09.zip
"Install Google Play Intel x86 Atom System Image API 28 (revision 9)" ready.
Installing Google Play Intel x86 Atom System Image in
C:\Users\mgit110-10\AppData\Local\Android\Sdk\system-images\android-28\google_apis_playstore\x86
"Install Google Play Intel x86 Atom System Image API 28 (revision 9)" complete.
"Install Google Play Intel x86 Atom System Image API 28 (revision 9)" finished.
Done
Previous
Next
Cancel
Finish
MainActivity.kt
Type here to search
BSE midcap +0.88%
3:29 PM
7/31/2024


lab manual - Microsoft Word
hello_1
Version control
No Devices
app
Virtual Device Configuration
System Image
Select a system image
Recommended
x86 Images
Other Images
Release Name
API
ABI
xABI
Target
Pie
28
x86
Android 9.0 (Google
API Level
28
Type
Google Play
Android
9.0
Google Inc.
Previous
Next
Cancel
Finish
MainActivity.kt
Type here to search
BSE midcap +0.88%
3:29 PM
7/31/2024


lab manual - Microsoft Word
hello_1
Version control
No Devices
app
Android
Virtual Device Configuration
Android Virtual Device (AVD)
Verify Configuration
AVD name:
Pixel 4 API 28
Pixel 4
5.7 1080x2280
Creating Android Virtual Device
Pie
Android 9.0 x8
Preferred ABI:
Optimal
Startup orientation:
Recommendation
HAXM is not installed.
Install HAXM
Show Advanced Settings
Previous
Next
Cancel
Finish
hello_1 > app > src > main > java > com > example > hello_1 > MainActivity.kt
1:1 LF UTF-8 4 spaces
Words: 0
100%
Type here to search
30°C Haze
ENG
IN
3:30 PM
7/31/2024

1 b) Write a simple Dart program to understand the language basics

```
void main() {
  // Define three static numbers
  int num1 = 10;
  int num2 = 20;
  int num3 = 30;

  // Calculate the sum of the three numbers
  int sum = num1 + num2 + num3;

  // Print the result
  print('The sum of $num1, $num2, and $num3 is $sum.');
}
```



2 a) Explore various Flutter widgets (Text, Image, Container, etc.).

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
        home: Scaffold(
      appBar: AppBar(
        title: Text('Image'),
      ),
      body: Center(
        child: Container(
          height: 200,
          width: 300,
          child: Image.network('img_src'),
        ),
      ),
    ));
  }
}
```

output:



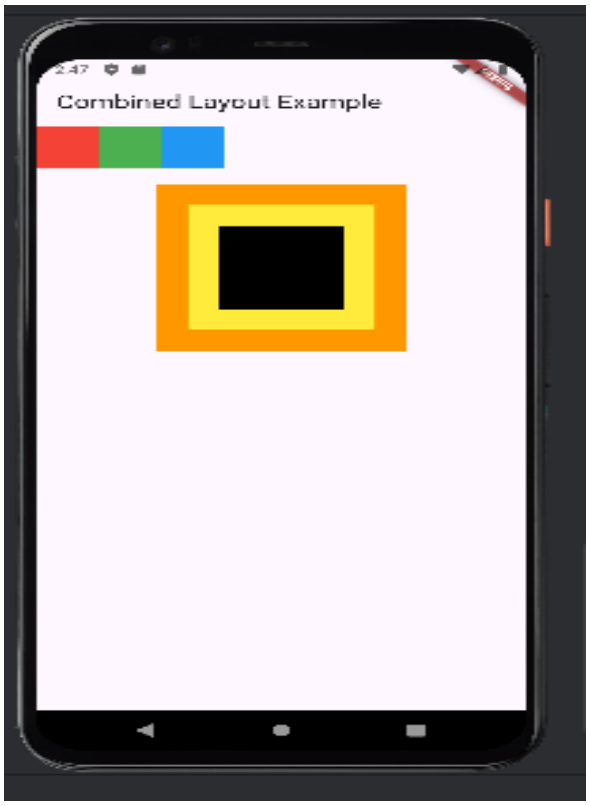
2 b) Implement different layout structures using Row, Column, and Stack widgets.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Combined Layout Example')),
```

```
        body: Center(
          child: Column(
            //mainAxisAlignment: MainAxisAlignment.center,
            children: <Widget>[
              Row(
                //mainAxisAlignment: MainAxisAlignment.spaceAround,
                children: <Widget>[
                  Container(color: Colors.red, width: 50, height: 50),
                  Container(color: Colors.green, width: 50, height: 50),
                  Container(color: Colors.blue, width: 50, height: 50),
                ],
              ),
              SizedBox(height: 20),
              Stack(
                alignment: Alignment.center,
                children: <Widget>[
                  Container(color: Colors.red, width: 200, height: 200),
                  Container(color: Colors.green, width: 150, height: 150),
                  Container(color: Colors.blue, width: 100, height: 100),
                ],
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```



3 a) Design a responsive UI that adapts to different screen sizes

```
import 'package:flutter/material.dart';

//main function
void main() {
  runApp(MyApp()); //leets change it
}

//stateless widget class
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return SafeArea(
      child: MaterialApp(
        home: Text('AIML'),
```

```
      ),
    );
  }
}
```



3 b) Implement media queries and breakpoints for responsiveness.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: ResponsiveLayout(),
```

```
    );
  }
}

class ResponsiveLayout extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var mediaQueryData = MediaQuery.of(context);
    var screenWidth = mediaQueryData.size.width;

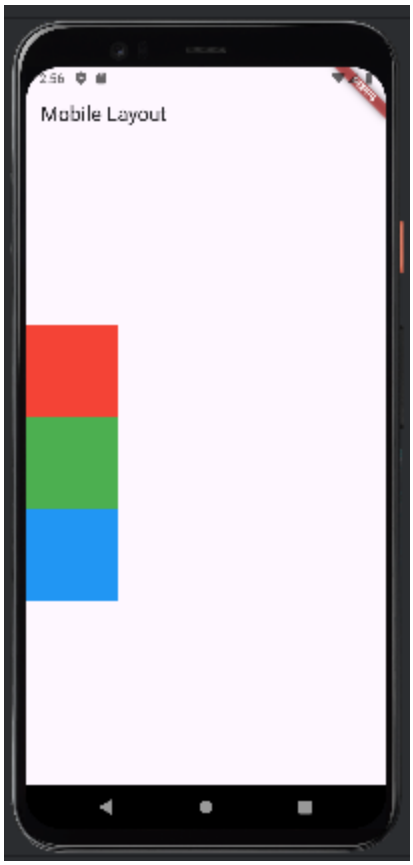
    if (screenWidth < 600) {
      return Scaffold(
        appBar: AppBar(title: Text('Mobile Layout')),
        body: _buildNarrowContainers(),
      );
    } else if (screenWidth < 1200) {
      return Scaffold(
        appBar: AppBar(title: Text('Tablet Layout')),
        body: _buildMediumContainers(),
      );
    } else {
      return Scaffold(
        appBar: AppBar(title: Text('Desktop Layout')),
        body: _buildWideContainers(),
      );
    }
  }

  Widget _buildNarrowContainers() {
    return Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: <Widget>[
        Container(color: Colors.red, width: 100, height: 100),
        Container(color: Colors.green, width: 100, height: 100),
        Container(color: Colors.blue, width: 100, height: 100),
```

```
      ],
    );
  }

  Widget _buildMediumContainers() {
    return Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: <Widget>[
        Container(color: Colors.red, width: 100, height: 100),
        Container(color: Colors.green, width: 100, height: 100),
        Container(color: Colors.blue, width: 100, height: 100),
      ],
    );
  }

  Widget _buildWideContainers() {
    return GridView.count(
      crossAxisCount: 3,
      mainAxisSpacing: 10,
      crossAxisSpacing: 10,
      children: <Widget>[
        Container(color: Colors.red, width: 100, height: 100),
        Container(color: Colors.green, width: 100, height: 100),
        Container(color: Colors.blue, width: 100, height: 100),
      ],
    );
  }
}
```



4 a) Set up navigation between different screens using Navigator.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

//1st MyApp class - hub class
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
      home: HomeScreen(),
    );
  }
}

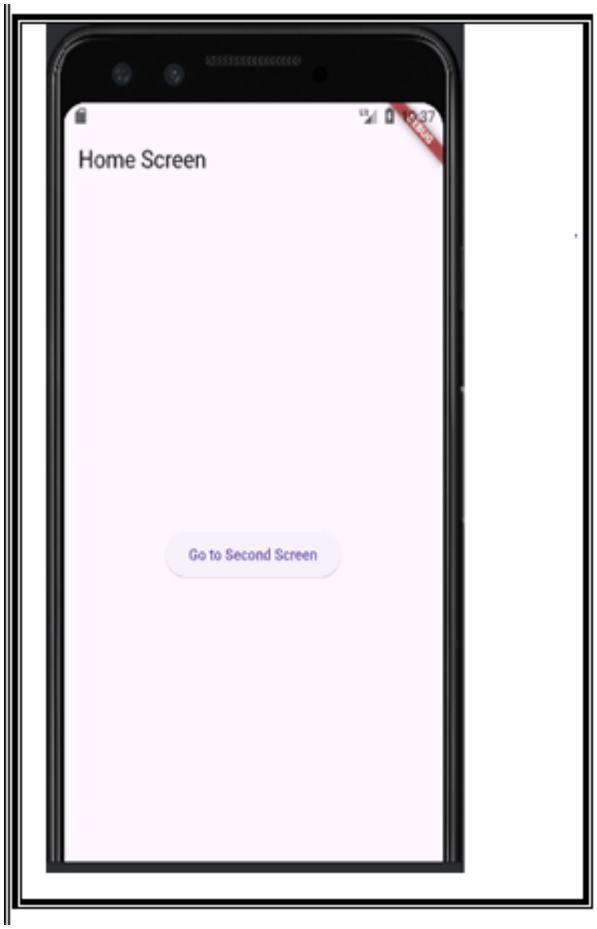
//Class HomeScreen- first page in app
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

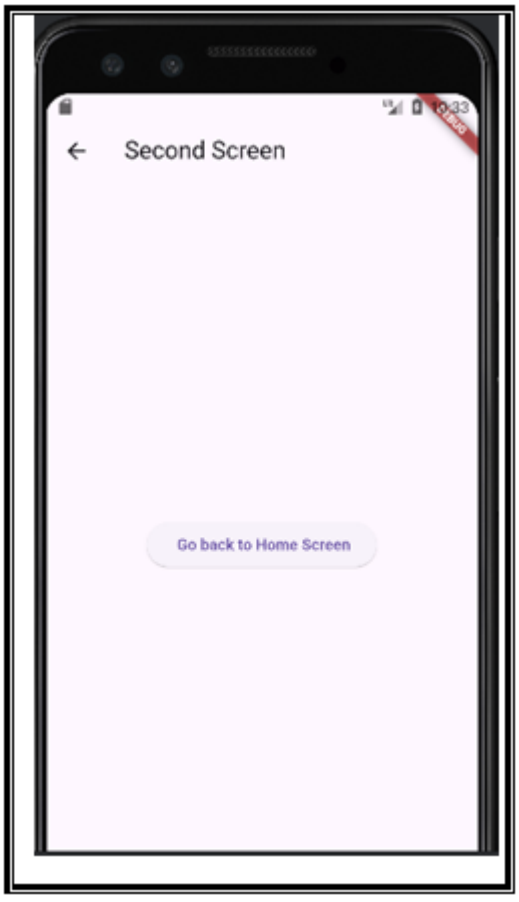
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Home screen'),
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => SecondScreen()),
            );
          },
          child: Text('Go to Second Screen'),
        ),
      ),
    );
  }
}

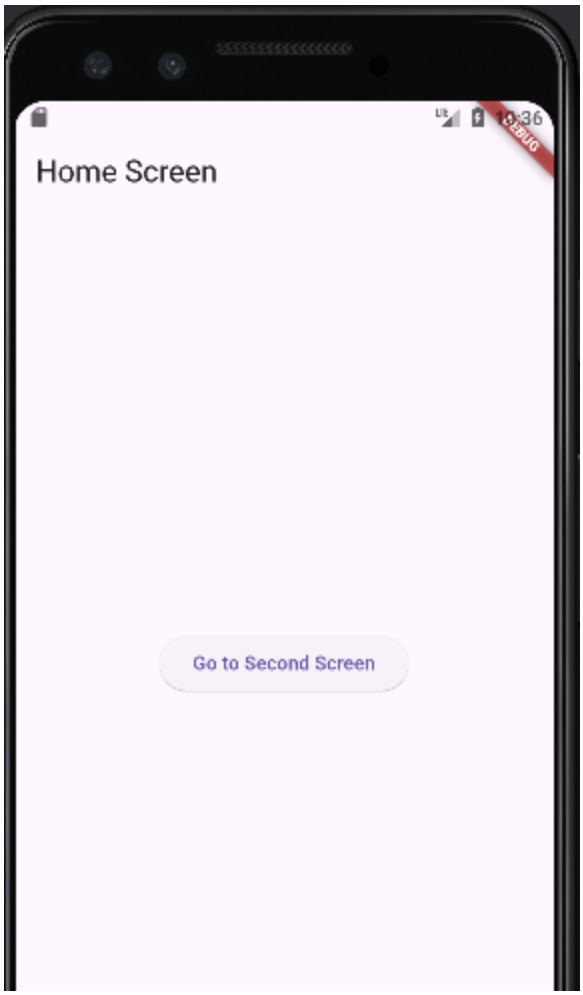
//3rd claas SecondScreen- second page
class SecondScreen extends StatelessWidget {
  const SecondScreen({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Second screen'),
      ),
      body: ElevatedButton(
        onPressed: () {
          Navigator.pop(context);
        },
        child: Text('Go back to Home screen'),
      ),
    );
  }
}
```






Home Screen
DEBUG
Go to Second Screen

4 b) Implement navigation with named routes.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

//1st MyApp class - hub class
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      initialRoute: '/',
      routes: {
        '/': (context) => HomeScreen(),
        '/second': (context) => SecondScreen(),
      },
    );
  }
}

//Class HomeScreen- first page in app
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

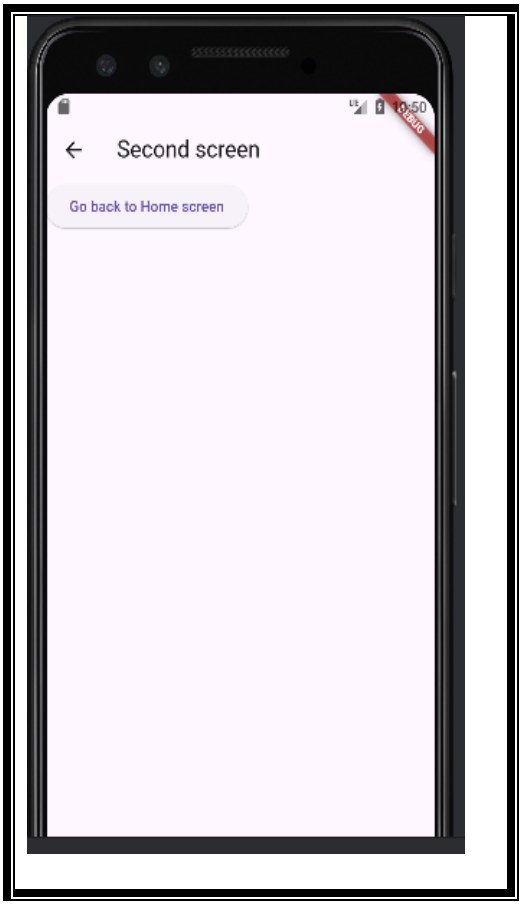
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Home screen'),
      ),
      body: Center(
```

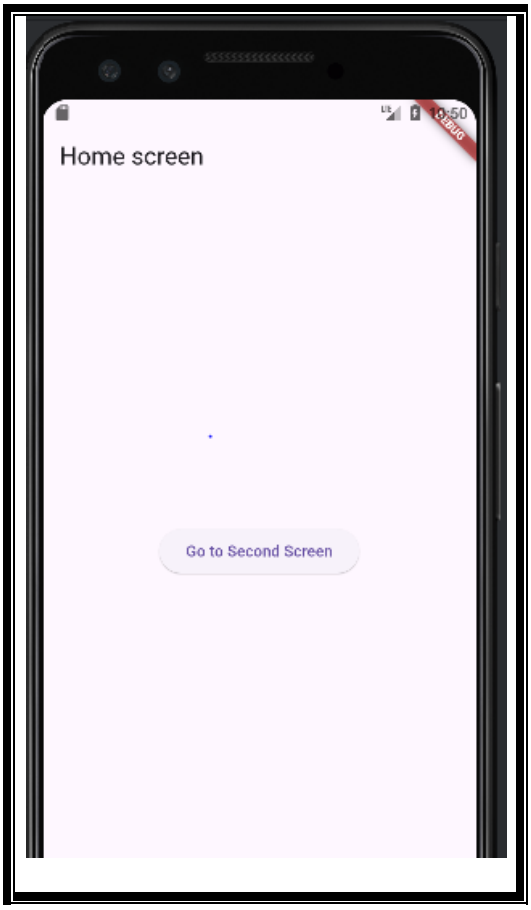
```
        child: ElevatedButton(
          onPressed: () {
            Navigator.pushNamed(context, '/second');
          },
          child: Text('Go to Second Screen'),
        ),
      ),
    );
  }
}

//3rd claas SecondScreen- second page
class SecondScreen extends StatelessWidget {
  const SecondScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Second screen'),
      ),
      body: ElevatedButton(
        onPressed: () {
          Navigator.pop(context);
        },
        child: Text('Go back to Home screen'),
      ),
    );
  }
```

}





5 a) Learn about stateful and stateless widgets

```
import 'package:flutter/material.dart';

//main function
void main() {
  runApp(MyApp());
}

class MyApp extends StatefulWidget {
  const MyApp({super.key});
  @override
  State<MyApp> createState() => _MyAppState();
}
```

```
class _MyAppState extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Column(
        children: [Text('data-1'), Text('data-2'), Text('data-3')],
      ),
    );
}
}
```



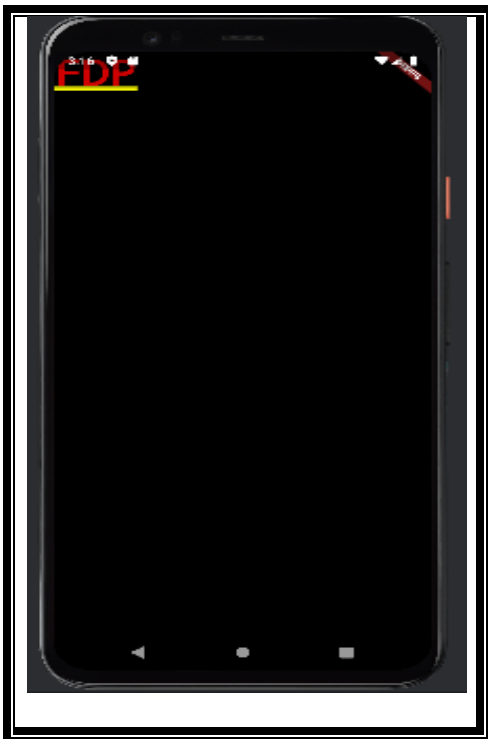
5 (A) (2)

```
import 'package:flutter/material.dart';

//main function
void main() {
  runApp(MyApp()); //leets change it
}

//stateless widget class
class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Text('FDP'),
    );
  }
}
```



5 b) Implement state management using set State and Provider.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  String value = "Test"; //variable initialized
  //fucntion to change the value
  void clickMe() {
    setState(() {
      value = "Test done";
    });
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('simple Flutter app'),
        ),
        body: Center(
          child: Column(
```

```
            children: [
              Text('$value'),
            ],
          ),
        ),
        floatingActionButton: FloatingActionButton(
          child: Icon(Icons.add),
          onPressed: clickMe,
        ),
      ),
    );
  }
}
```



6 a) Create custom widgets for specific UI elements.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: HomeScreen(),
    );
  }
}

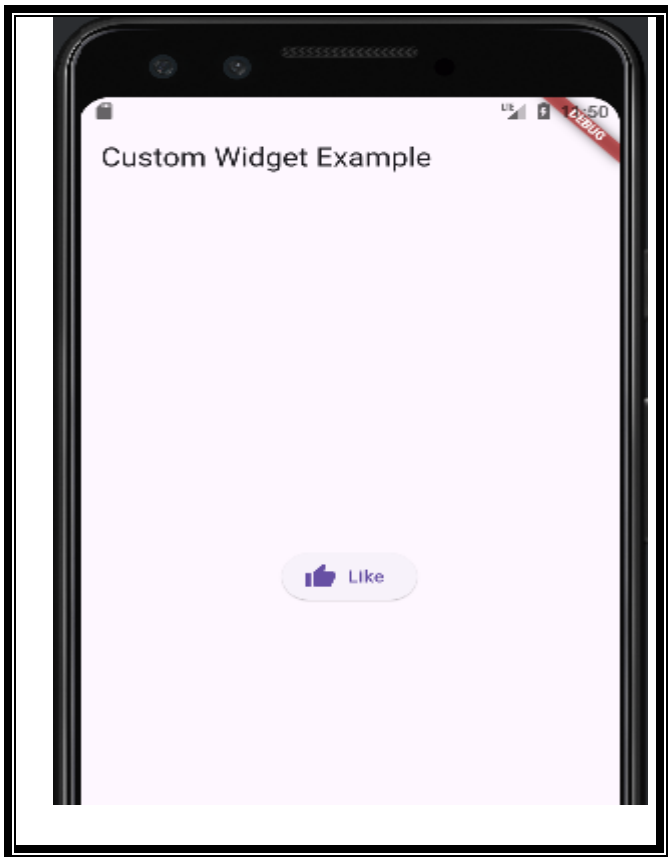
//Home screen - class
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

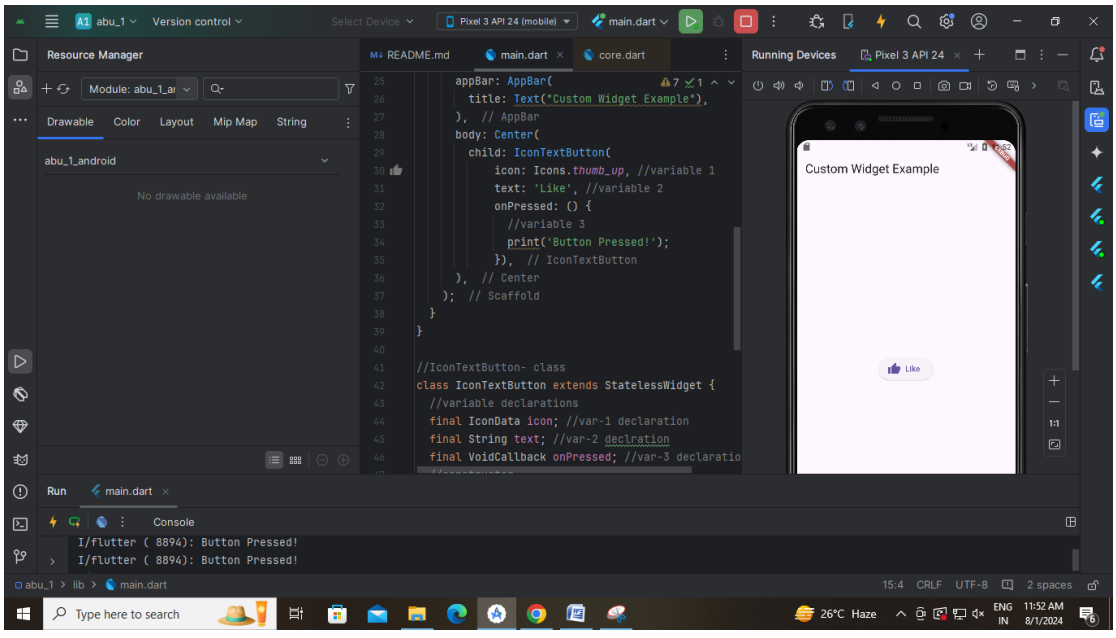
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Custom Widget Example"),
      ),
      body: Center(
        child: IconTextButton(
            icon: Icons.thumb_up, //variable 1
            text: 'Like', //variable 2
            onPressed: () {
              //variable 3
```

```
            print('Button Pressed!');
          }),
      ),
    );
  }
}

//IconTextButton- class
class IconTextButton extends StatelessWidget {
  //variable declarations
  final IconData icon; //var-1 declaration
  final String text; //var-2 declration
  final VoidCallback onPressed; //var-3 declaration
  //constructor
  IconTextButton({
    required this.icon,
    required this.text,
    required this.onPressed,
  });

  @override
  Widget build(BuildContext context) {
    return ElevatedButton.icon(
      onPressed: onPressed,
      icon: Icon(icon),
      label: Text(text),
    );
  }
}
```


Custom Widget Example
Like


Resource Manager
README.md
main.dart
core.dart
Running Devices
Pixel 3 API 24
Custom Widget Example
Like
Run
Console
I/flutter (8894): Button Pressed!
I/flutter (8894): Button Pressed!

6 b) Apply styling using themes and custom styles.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: HomeScreen(),
    );
  }
}

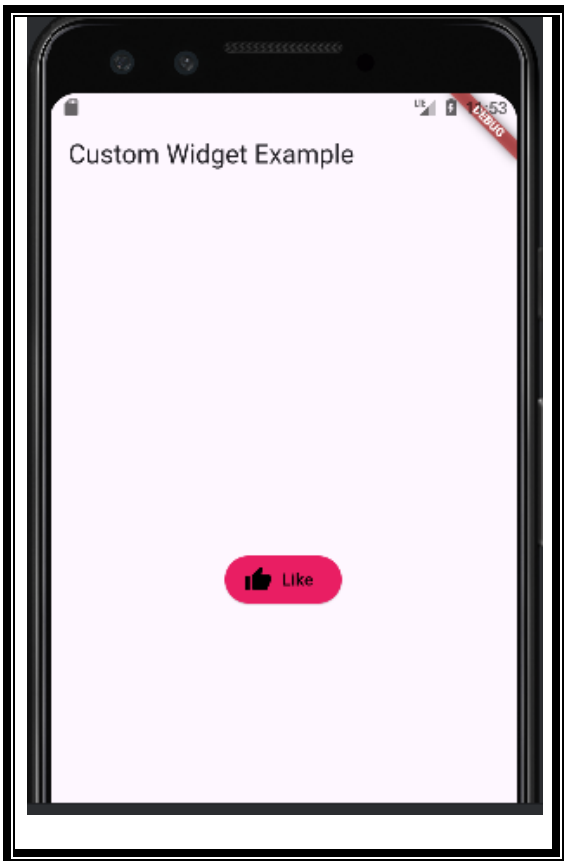
//Home screen - class
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

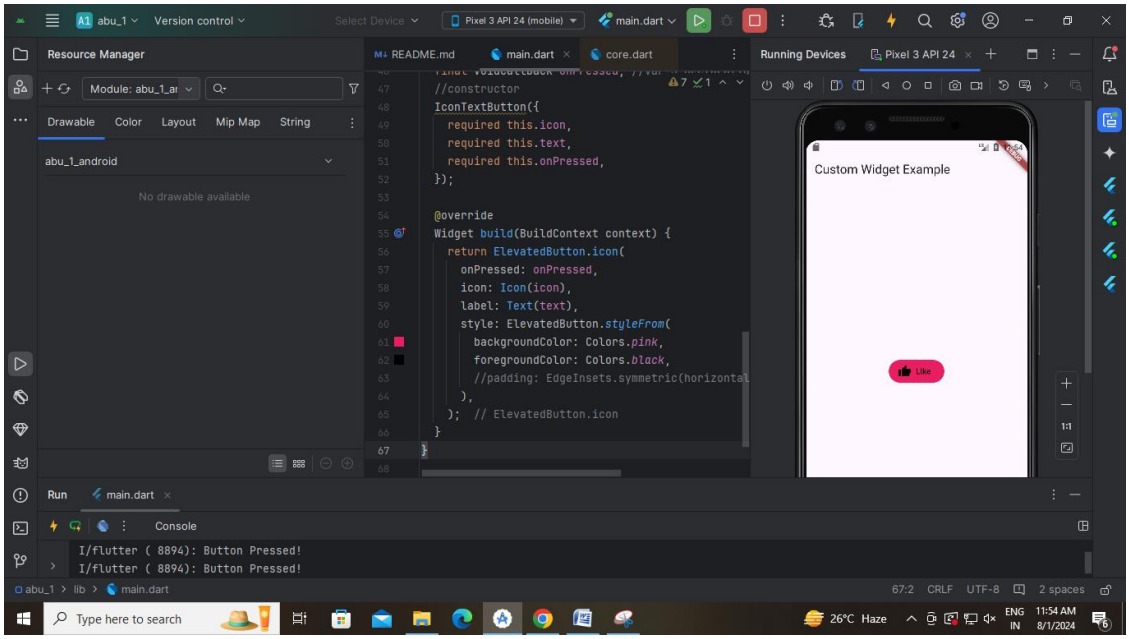
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Custom Widget Example"),
      ),
      body: Center(
        child: IconTextButton(
            icon: Icons.thumb_up, //variable 1
            text: 'Like', //variable 2
            onPressed: () {
```

```
            //variable 3
            print('Button Pressed!');
          }),
      ),
    );
  }
}

//IconTextButton- class
class IconTextButton extends StatelessWidget {
  //variable declarations
  final IconData icon; //var-1 declaration
  final String text; //var-2 declration
  final VoidCallback onPressed; //var-3 declaration
  //constructor
  IconTextButton({
    required this.icon,
    required this.text,
    required this.onPressed,
  });

  @override
  Widget build(BuildContext context) {
    return ElevatedButton.icon(
      onPressed: onPressed,
      icon: Icon(icon),
      label: Text(text),
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.pink,
        foregroundColor: Colors.black,
        //padding: EdgeInsets.symmetric(horizontal: 16.0, vertical: 8.0),
      ),
    );
  }
}
```


Custom Widget Example
Like


Resource Manager
README.md
main.dart
core.dart
Running Devices
Pixel 3 API 24
Custom Widget Example
Like
Run
Console
I/flutter (8894): Button Pressed!
I/flutter (8894): Button Pressed!

7 a) Design a form with various input fields.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

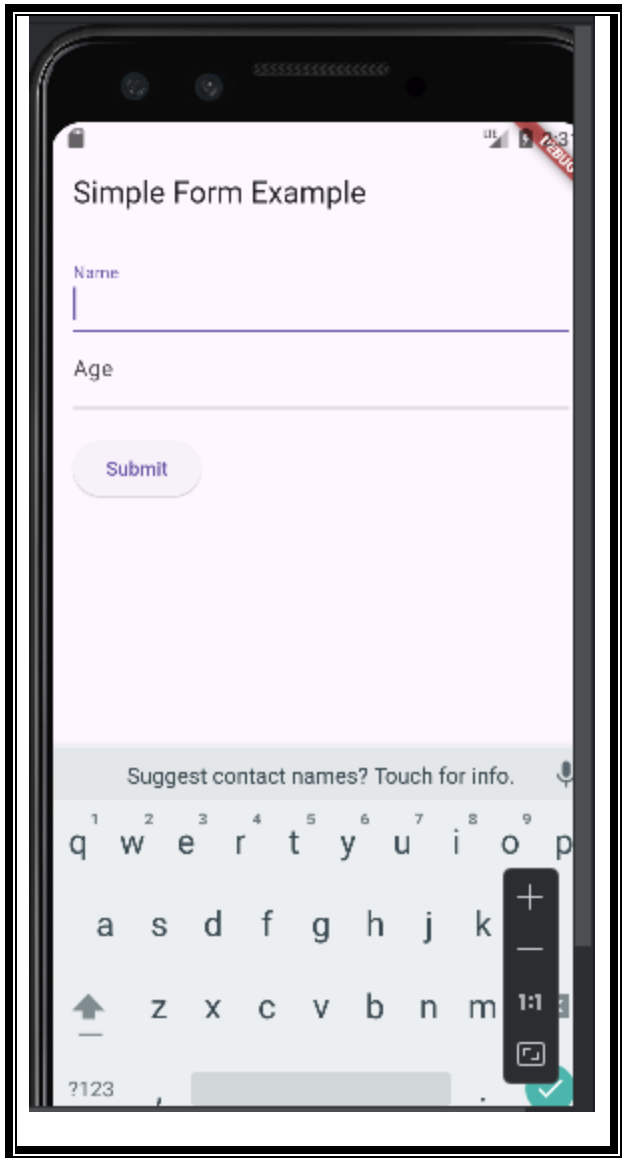
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Simple Form Example'),
        ),
        body: Padding(
          padding: const EdgeInsets.all(16.0),
          child: SimpleForm(),
        ),
      ),
    );
  }
}

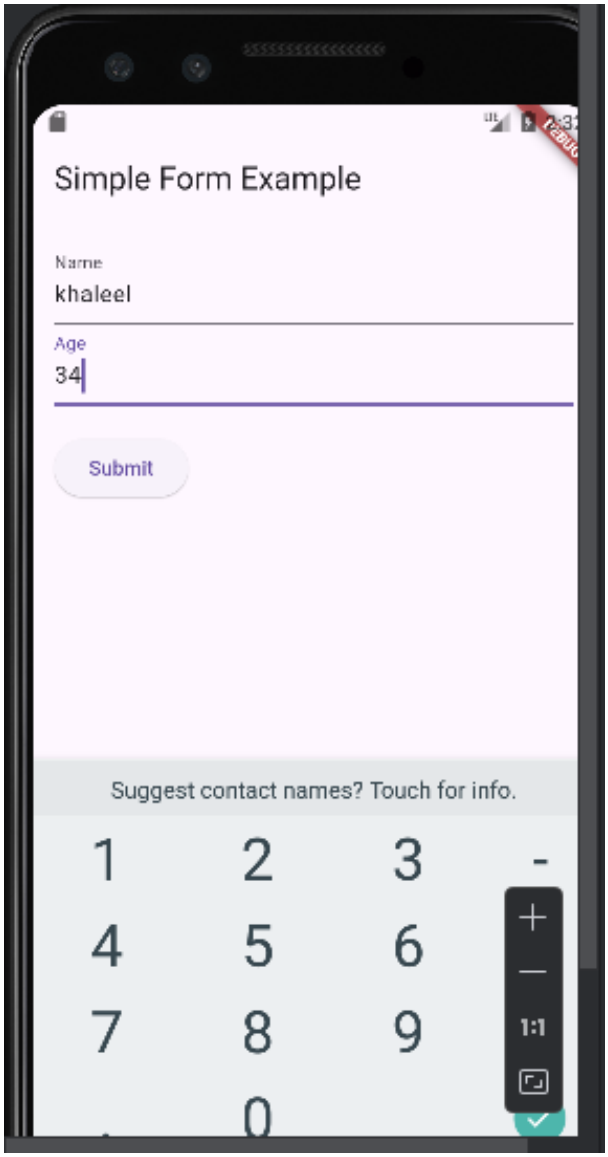
class SimpleForm extends StatefulWidget {
  @override
  _SimpleFormState createState() => _SimpleFormState();
}

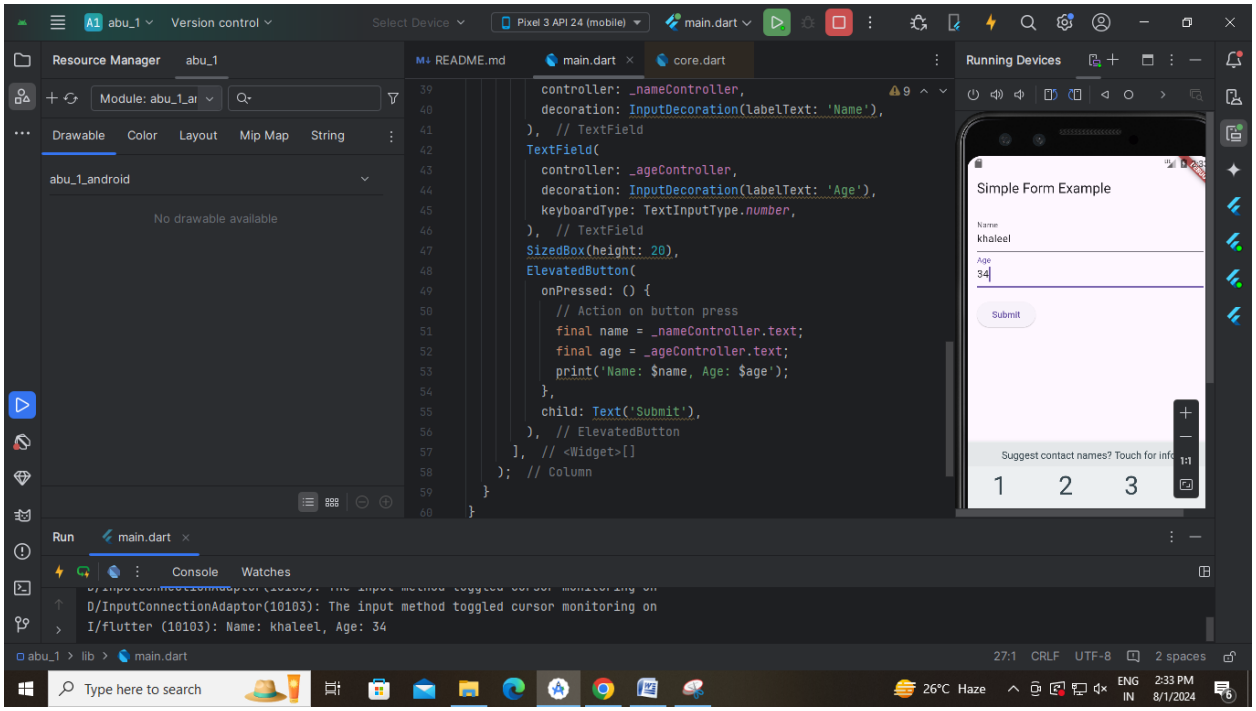
class _SimpleFormState extends State<SimpleForm> {
  final _nameController = TextEditingController();
  final _ageController = TextEditingController();

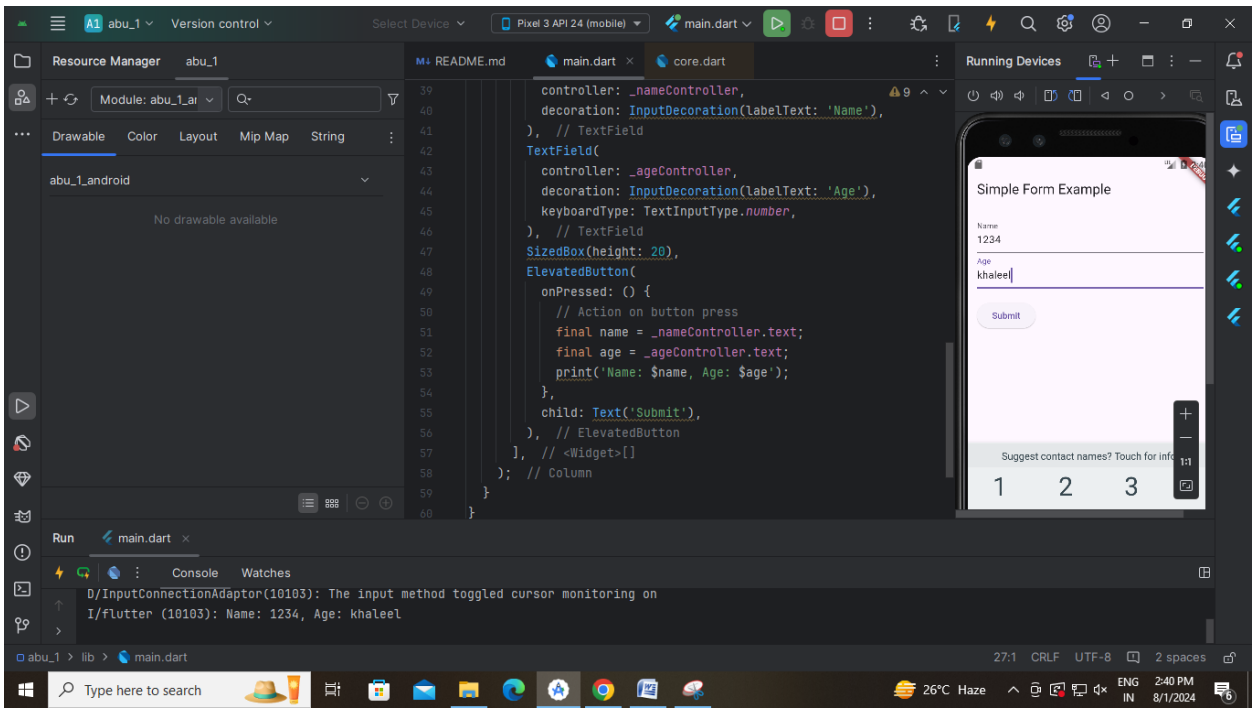
  @override
```

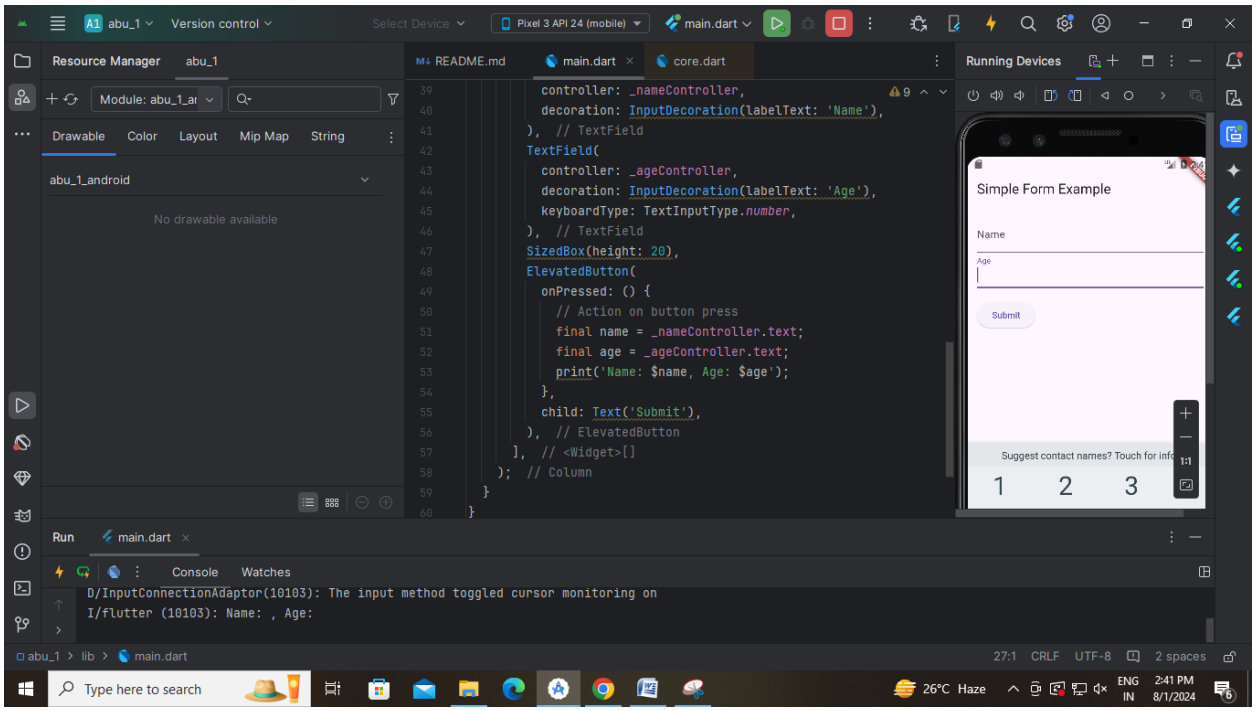
```
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: <Widget>[
        TextField(
          controller: _nameController,
          decoration: InputDecoration(labelText: 'Name'),
        ),
        TextField(
          controller: _ageController,
          decoration: InputDecoration(labelText: 'Age'),
          keyboardType: TextInputType.number,
        ),
        SizedBox(height: 20),
        ElevatedButton(
          onPressed: () {
            // Action on button press
            final name = _nameController.text;
            final age = _ageController.text;
            print('Name: $name, Age: $age');
          },
          child: Text('Submit'),
        ),
      ],
    );
  }
}
```


Simple Form Example
Name
Age
Submit
Suggest contact names? Touch for info.
q w e r t y u i o p
a s d f g h j k
z x c v b n m
?123


Simple Form Example
Name
khaleel
Age
34
Submit
Suggest contact names? Touch for info.
1
2
3
-
4
5
6
7
8
9
1:1
0


abu_1
Version control
Pixel 3 API 24 (mobile)
main.dart
Resource Manager
abu_1
README.md
main.dart
core.dart
Running Devices
Module: abu_1_a
Drawable
Color
Layout
Mip Map
String
abu_1_android
No drawable available
controller: _nameController,
decoration: InputDecoration(labelText: 'Name'),
), // TextField
TextField(
controller: _ageController,
decoration: InputDecoration(labelText: 'Age'),
keyboardType: TextInputType.number,
), // TextField
SizedBox(height: 20),
ElevatedButton(
onPressed: () {
// Action on button press
final name = _nameController.text;
final age = _ageController.text;
print('Name: $name, Age: $age');
},
child: Text('Submit'),
), // ElevatedButton
], // <Widget>[]
); // Column
}
}
Simple Form Example
Name
khaleel
Age
34
Submit
Suggest contact names? Touch for info
1 2 3
Run
main.dart
Console
Watches
D/InputConnectionAdaptor(10103): The input method toggled cursor monitoring on
I/flutter (10103): Name: khaleel, Age: 34
abu_1 > lib > main.dart
27:1 CRLF UTF-8 2 spaces
Type here to search
26°C Haze
ENG IN
2:33 PM 8/1/2024


abu_1
Version control
Pixel 3 API 24 (mobile)
main.dart
Resource Manager
abu_1
README.md
main.dart
core.dart
Running Devices
Module: abu_1_a
Drawable
Color
Layout
Mip Map
String
abu_1_android
No drawable available
controller: _nameController,
decoration: InputDecoration(labelText: 'Name'),
), // TextField
TextField(
controller: _ageController,
decoration: InputDecoration(labelText: 'Age'),
keyboardType: TextInputType.number,
), // TextField
SizedBox(height: 20),
ElevatedButton(
onPressed: () {
// Action on button press
final name = _nameController.text;
final age = _ageController.text;
print('Name: $name, Age: $age');
},
child: Text('Submit'),
), // ElevatedButton
], // <Widget>[]
); // Column
}
}
Simple Form Example
Name
1234
Age
khaleel
Submit
Suggest contact names? Touch for info
1 2 3
Run
main.dart
Console
Watches
D/InputConnectionAdaptor(10103): The input method toggled cursor monitoring on
I/flutter (10103): Name: 1234, Age: khaleel
abu_1 > lib > main.dart
27:1 CRLF UTF-8 2 spaces
Type here to search
26°C Haze
ENG IN
2:40 PM 8/1/2024



7 b) Implement form validation and error handling.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Simple Form with Validation'),
        ),
        body: Padding(
```

```
          padding: const EdgeInsets.all(16.0),
          child: SimpleForm(),
        ),
      ),
    );
  }
}

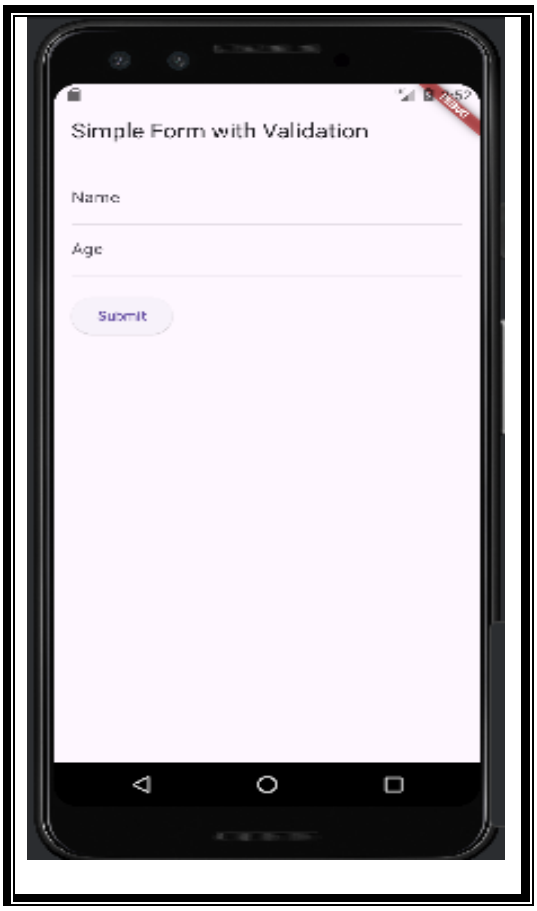
class SimpleForm extends StatefulWidget {
  @override
  _SimpleFormState createState() => _SimpleFormState();
}

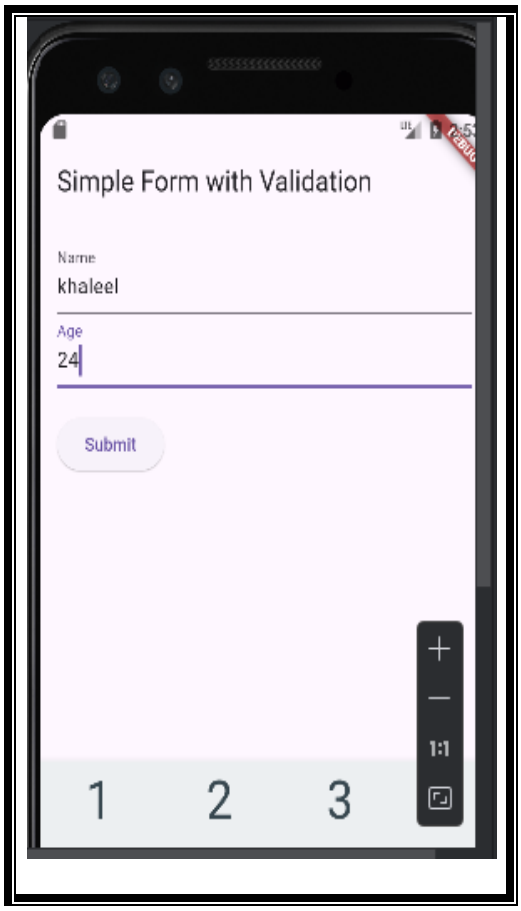
class _SimpleFormState extends State<SimpleForm> {
  final _formKey = GlobalKey<FormState>();

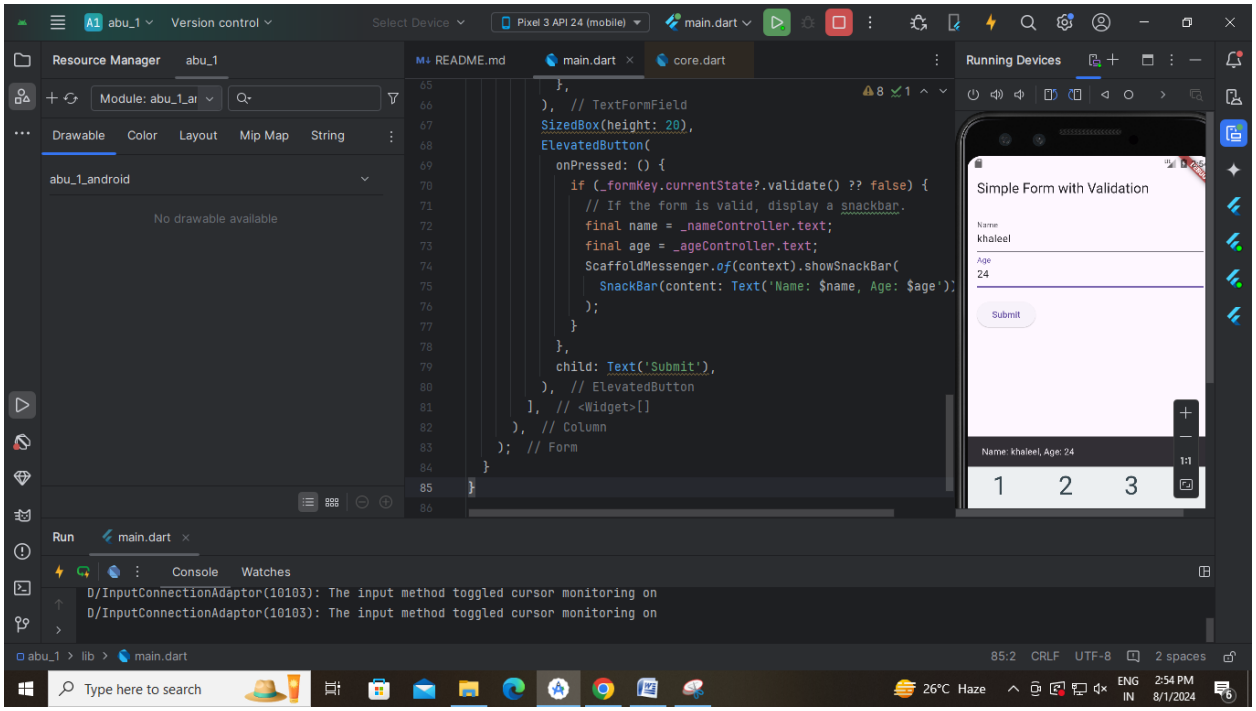
  final _nameController = TextEditingController();
  final _ageController = TextEditingController();

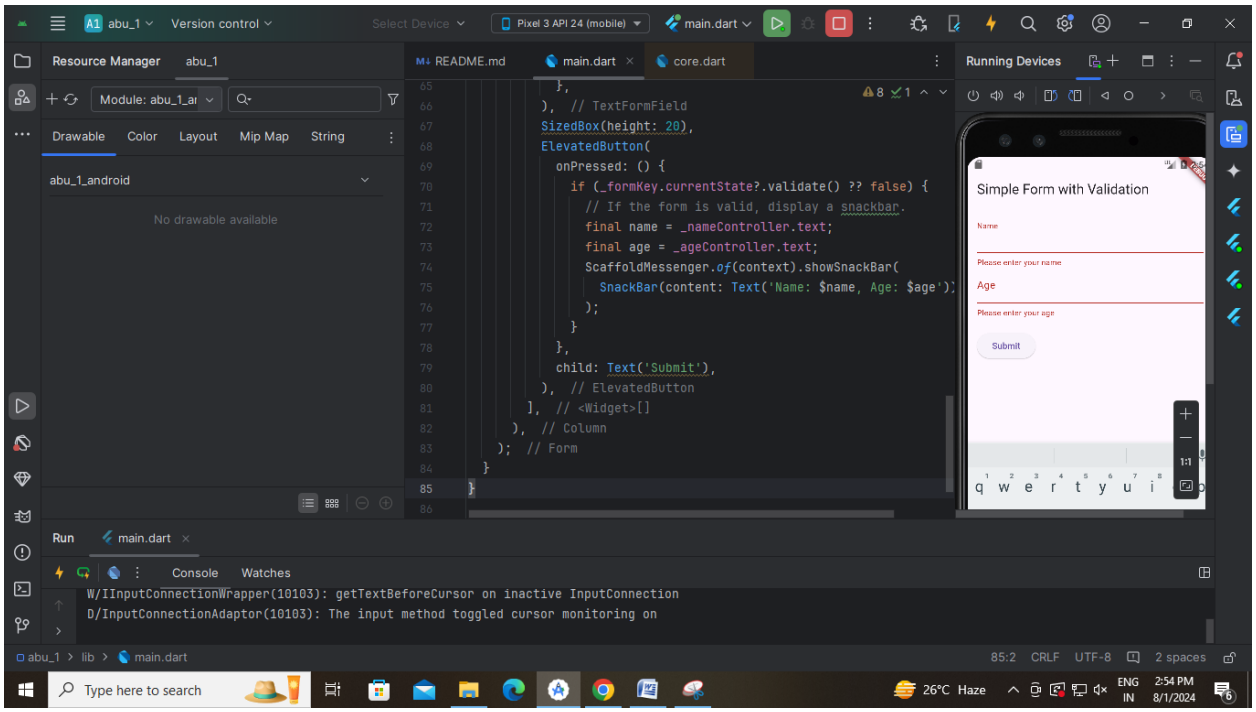
  @override
  Widget build(BuildContext context) {
    return Form(
      key: _formKey,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          TextFormField(
            controller: _nameController,
            decoration: InputDecoration(labelText: 'Name'),
            validator: (value) {
              if (value == null || value.isEmpty) {
                return 'Please enter your name';
              }
              return null;
            },
```

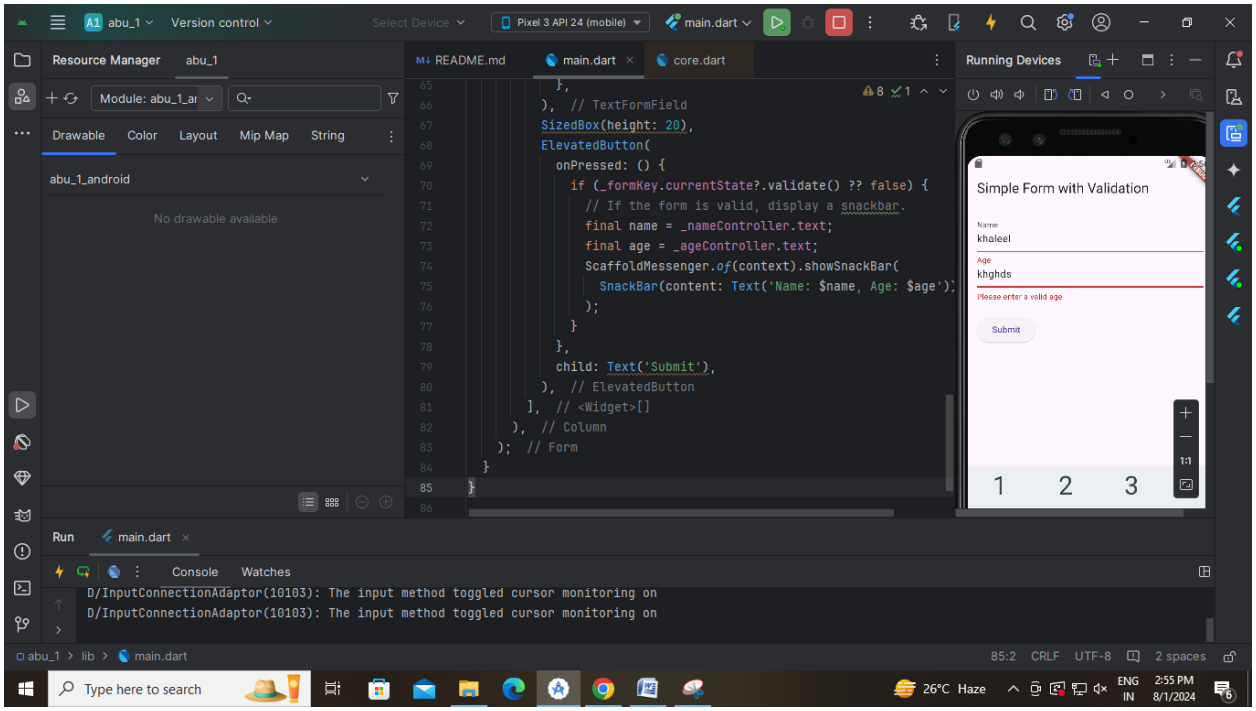
```
          ),
          TextFormField(
            controller: _ageController,
            decoration: InputDecoration(labelText: 'Age'),
            keyboardType: TextInputType.number,
            validator: (value) {
              if (value == null || value.isEmpty) {
                return 'Please enter your age';
              }
              final age = int.tryParse(value);
              if (age == null || age <= 0) {
                return 'Please enter a valid age';
              }
              return null;
            },
          ),
          SizedBox(height: 20),
          ElevatedButton(
            onPressed: () {
              if (_formKey.currentState?.validate() ?? false) {
                // If the form is valid, display a snackbar.
                final name = _nameController.text;
                final age = _ageController.text;
                ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(content: Text('Name: $name, Age: $age')),
                );
              }
            },
            child: Text('Submit'),
          ),
        ],
      ),
    );
  }
}
```

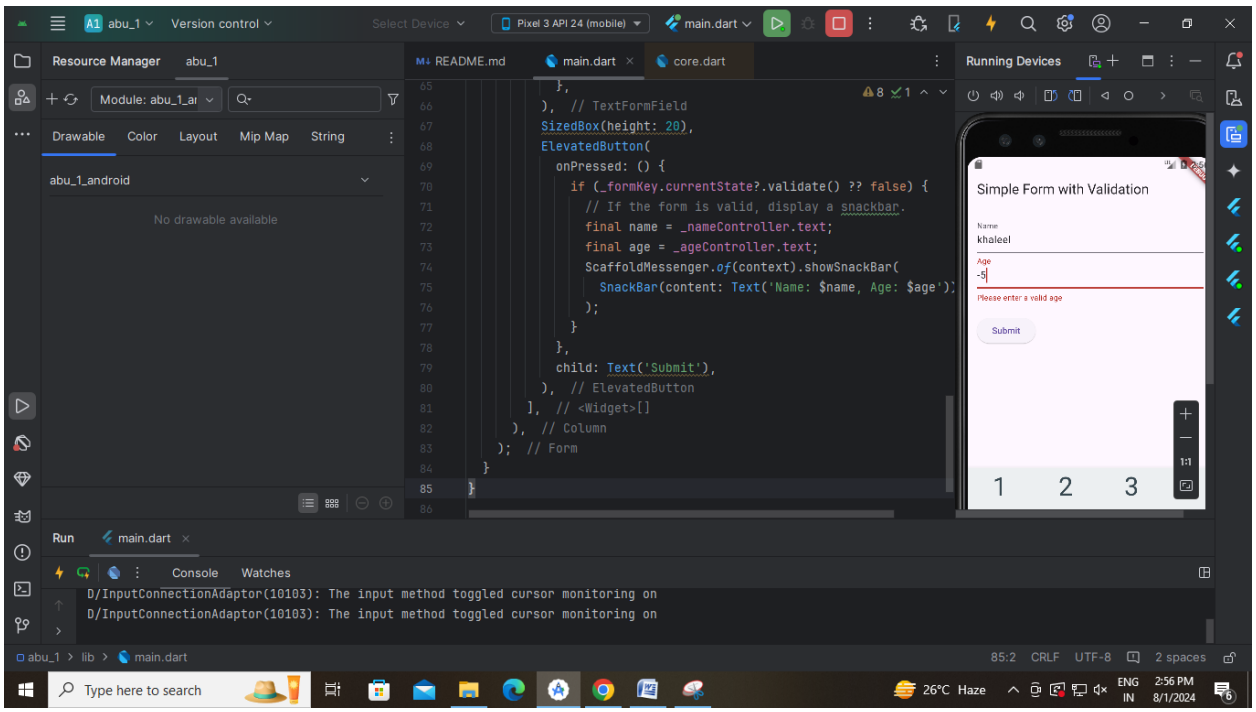

Simple Form with Validation
Name
Age
Submit

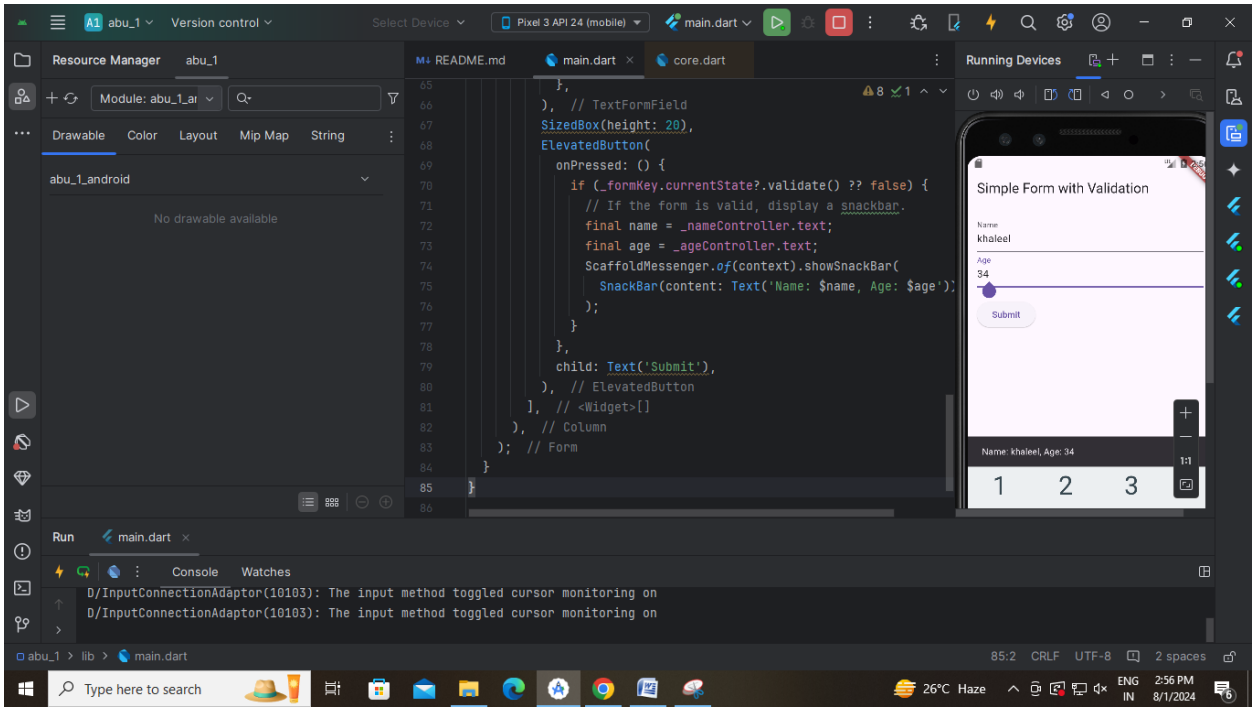

Simple Form with Validation
Name
khaleel
Age
24
Submit
1:1
1
2
3


Resource Manager
abu_1
README.md
main.dart
core.dart
Running Devices
Simple Form with Validation
Name
khaleel
Age
24
Submit
Name: khaleel, Age: 24
Run
main.dart
Console
Watches
D/InputConnectionAdaptor(10103): The input method toggled cursor monitoring on
D/InputConnectionAdaptor(10103): The input method toggled cursor monitoring on


Resource Manager
abu_1
README.md
main.dart
core.dart
Running Devices
Simple Form with Validation
Name
Please enter your name
Age
Please enter your age
Submit
Run
main.dart
Console
Watches
W/IInputConnectionWrapper(10103): getTextBeforeCursor on inactive InputConnection
D/InputConnectionAdaptor(10103): The input method toggled cursor monitoring on


abu_1
Version control
Pixel 3 API 24 (mobile)
main.dart
Resource Manager
abu_1
README.md
main.dart
core.dart
Running Devices
Module: abu_1_ar
Drawable
Color
Layout
Mip Map
String
abu_1_android
No drawable available
Simple Form with Validation
Name
khaleel
Age
khghds
Please enter a valid age
Submit
1
2
3
Run
main.dart
Console
Watches
D/InputConnectionAdaptor(10103): The input method toggled cursor monitoring on
D/InputConnectionAdaptor(10103): The input method toggled cursor monitoring on
abu_1 > lib > main.dart
85:2
CRLF
UTF-8
2 spaces
Type here to search
26°C Haze
ENG
IN
2:55 PM
8/1/2024


abu_1
Version control
Pixel 3 API 24 (mobile)
main.dart
Resource Manager
abu_1
README.md
main.dart
core.dart
Running Devices
Module: abu_1_ar
Drawable
Color
Layout
Mip Map
String
abu_1_android
No drawable available
Simple Form with Validation
Name
khaleel
Age
-5
Please enter a valid age
Submit
1
2
3
Run
main.dart
Console
Watches
D/InputConnectionAdaptor(10103): The input method toggled cursor monitoring on
D/InputConnectionAdaptor(10103): The input method toggled cursor monitoring on
abu_1 > lib > main.dart
85:2
CRLF
UTF-8
2 spaces
Type here to search
26°C Haze
ENG
IN
2:56 PM
8/1/2024



8 a) Add animations to UI elements using Flutter's animation framework.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return MaterialApp(
      home: HomeScreen(),
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Custon widget with animation"),
      ),
      body: Center(
        child: AnimatedIconTextButton(
            icon: Icons.thumb_up,
            text: 'Like',
            onPressed: () {
              print("Button Pressed!");
            }),
      ),
    );
  }
}

class AnimatedIconTextButton extends StatefulWidget {
  final IconData icon;
  final String text;
  final VoidCallback onPressed;
  //constructors
  const AnimatedIconTextButton({
    Key? key,
```

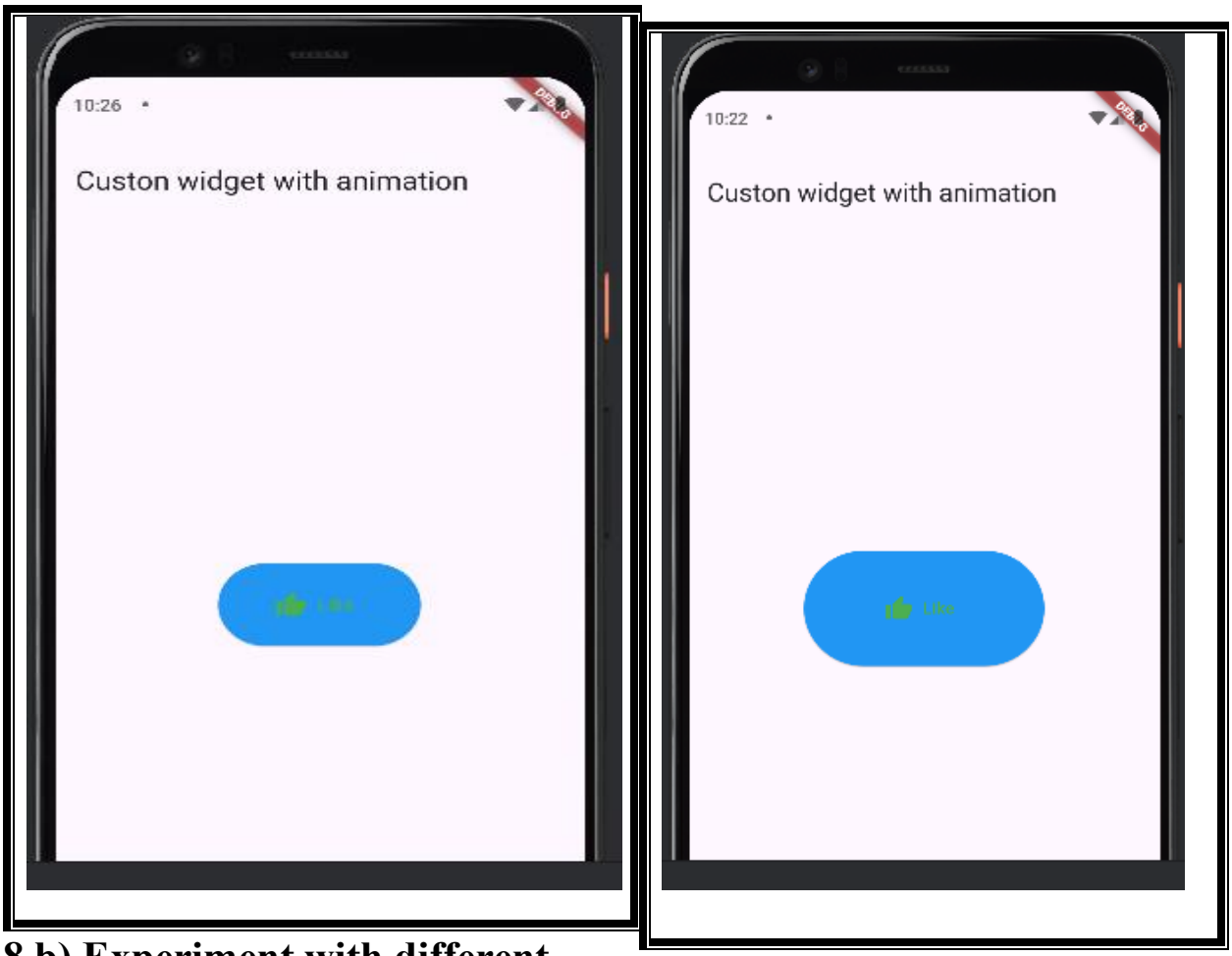
```
    required this.icon,
    required this.text,
    required this.onPressed,
  }) : super(key: key);

  @override
  State<AnimatedIconTextButton> createState() =>
_AnimatedIconTextButtonState();
}

class _AnimatedIconTextButtonState extends
State<AnimatedIconTextButton> {
  bool _isPressed = false;
  void _handlePress() {
    setState(() {
      _isPressed = !_isPressed; //on->off//off->on
    });
    widget.onPressed();
  }

  @override
  Widget build(BuildContext context) {
    return AnimatedContainer(
      duration: Duration(milliseconds: 300),
      width: _isPressed ? 200 : 150,
      height: _isPressed ? 100 : 70,
      child: ElevatedButton.icon(
        onPressed: _handlePress,
        icon: Icon(widget.icon),
        label: Text(widget.text),
        style: ElevatedButton.styleFrom(
          backgroundColor: Colors.blue,
          foregroundColor: Colors.green,
        ),
```

```
    ),
  );  }  }
```



8 b) Experiment with different types of animations (fade, slide)

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: HomeScreen(),
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Custon widget with animation"),
      ),
      body: Center(
        child: FadeIconTextButton(
            icon: Icons.thumb_up,
            text: 'Like',
            onPressed: () {
              print("Button Pressed!");
            }),
      ),
    );
  }
}

class FadeIconTextButton extends StatefulWidget {
  final IconData icon;
  final String text;
  final VoidCallback onPressed;
  //constructors
```

```
  const FadeIconTextButton({
    Key? key,
    required this.icon,
    required this.text,
    required this.onPressed,
  }) : super(key: key);

  @override
  State<FadeIconTextButton> createState() =>
_FadeIconTextButtonState();
}

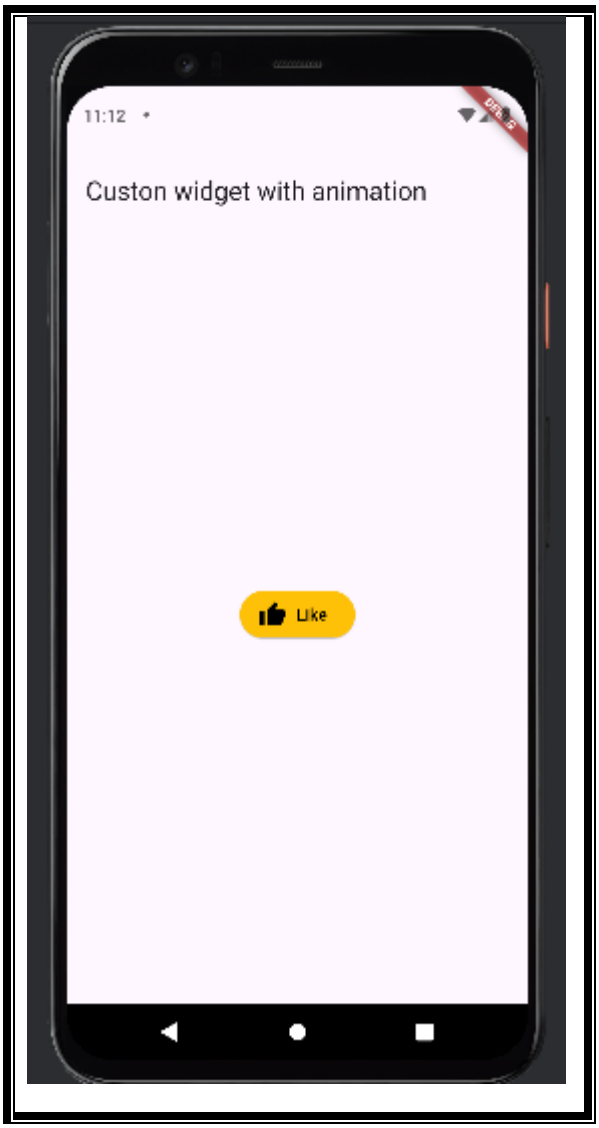
class _FadeIconTextButtonState extends State<FadeIconTextButton>
    with SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<double> _animation;
  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      duration: const Duration(milliseconds: 500),
      vsync: this,
    );
    _animation = CurvedAnimation(
      parent: _controller,
      curve: Curves.easeInOut,
    );
  }

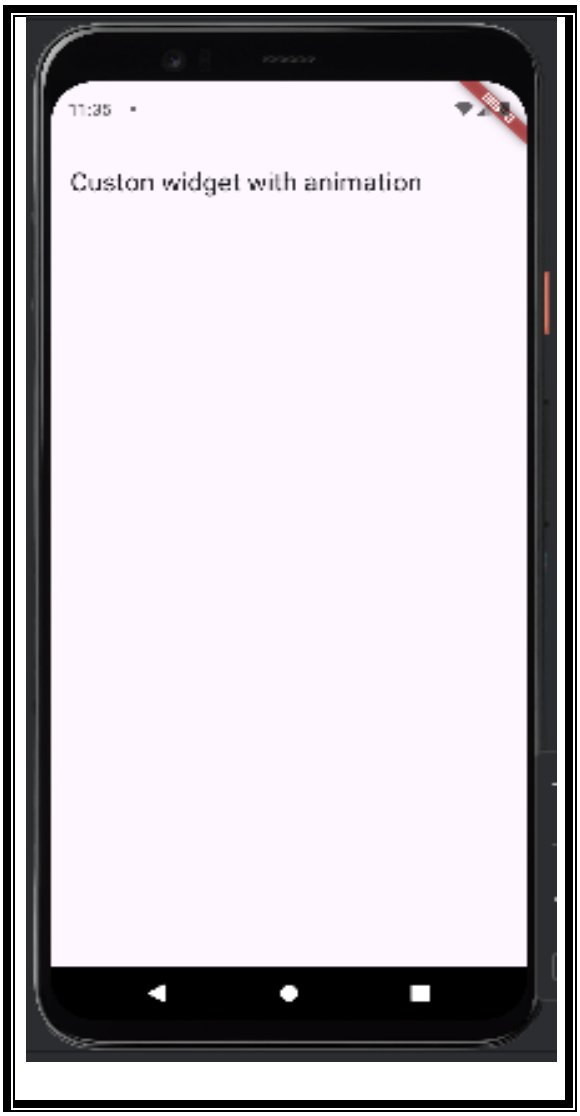
  void _handlePress() {
    if (_controller.isCompleted) {
      _controller.reverse();
    } else {
      _controller.forward();
    }
```

```
    widget.onPressed();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return FadeTransition(
      opacity: _animation,
      child: ElevatedButton.icon(
        onPressed: _handlePress,
        icon: Icon(widget.icon),
        label: Text(widget.text),
        style: ElevatedButton.styleFrom(
          backgroundColor: Colors.amber,
          foregroundColor: Colors.black,
        ),
      ),
    );
  }
}
```


11:12
Custon widget with animation
Like



9 a) Fetch data from a REST API. b) Display the fetched data in a meaningful way in the UI.

```dart
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: HomeScreen(),
    );
  }
}

class HomeScreen extends StatefulWidget {
  @override
  _HomeScreenState createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  late Future<List<Post>> _futurePosts;

  @override
  void initState() {
    super.initState();
    _futurePosts = fetchPosts();
  }

  Future<List<Post>> fetchPosts() async {
    final response =
```

```
      await
http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts'));

  if (response.statusCode == 200) {
    List jsonResponse = json.decode(response.body);
    return jsonResponse.map((post) => Post.fromJson(post)).toList();
  } else {
    throw Exception('Failed to load posts');
  }
}

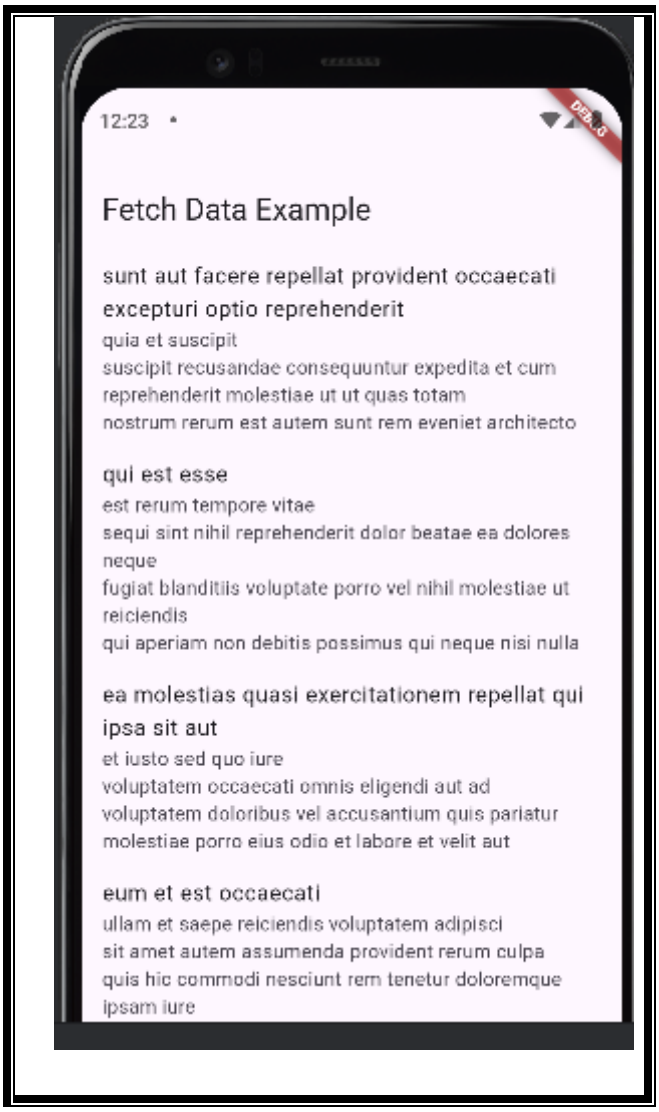
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Fetch Data Example'),
    ),
    body: Center(
      child: FutureBuilder<List<Post>>(
        future: _futurePosts,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return CircularProgressIndicator();
          } else if (snapshot.hasError) {
            return Text('${snapshot.error}');
          } else if (snapshot.hasData) {
            List<Post>? posts = snapshot.data;
            return ListView.builder(
              itemCount: posts!.length,
              itemBuilder: (context, index) {
                return ListTile(
                  title: Text(posts[index].title),
                  subtitle: Text(posts[index].body),
                );
              },
```

```
            );
          } else {
            return Text('No data found');
          }
        },
      ),
    ),
  );
 }
}

class Post {
 final int id;
 final String title;
 final String body;

 Post({required this.id, required this.title, required this.body});

 factory Post.fromJson(Map<String, dynamic> json) {
  return Post(
   id: json['id'],
   title: json['title'],
   body: json['body'],
  );
 }
}
```



10 a) Write unit tests for UI components.

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: HomeScreen(),
    );
  }
}

class HomeScreen extends StatefulWidget {
  @override
  _HomeScreenState createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  String _displayText = 'Hello';

  void _updateText() {
    setState(() {
      _displayText = 'Button Pressed';
    });
  }

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(
        title: Text('Home Screen'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            Text(_displayText),
            SizedBox(height: 20),
            ElevatedButton(
              onPressed: _updateText,
              child: Text('Press Me'),
            ),
          ],
        ),
      ),
    );
  }
}
```

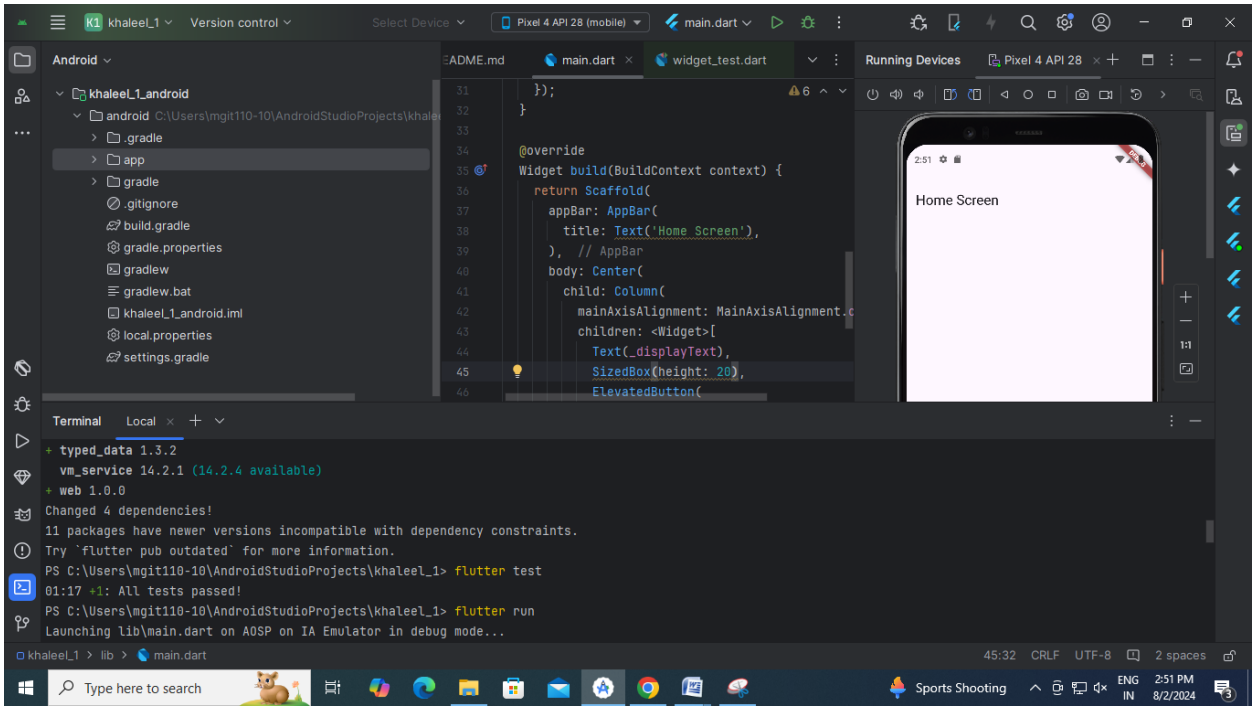
// search" Widget_test.dart" file by pressing Ctrl+Shift+f and past the bellow code

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:untitled/main.dart'; //url might change

void main() {
  testWidgets('text changes', (WidgetTester tester) async {
    //build the app
    await tester.pumpWidget(MyApp());
    //test-1 verifying the initial text'Hello
    expect(find.text("Hello"), findsOneWidget);
    expect(find.text("Button Pressed"), findsNothing);
```

```
    //test-2 to tap the button
    await tester.tap(find.byType(ElevatedButton));
    await tester.pump();
    //test-3 verify the text
    expect(find.text("Hello"), findsNothing);
    expect(find.text("Button Pressed"), findsOneWidget);
  });
}
```

- open terminal and type the following command
- flutter test
- it should show +1: All test passed



10 b) Use Flutter's debugging tools to identify and fix issues.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: HomeScreen(),
    );
  }
}

class HomeScreen extends StatefulWidget {
  @override
  _HomeScreenState createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  String _displayText; // Variable declared but not initialized

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Home Screen'),
      ),
      body: Center(
        child: Column(
```

```
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          Text(_displayText), // Error: _displayText is used before being
initialized
          SizedBox(height: 20),
          ElevatedButton(
            onPressed: () {
              setState(() {
                _displayText = 'Button Pressed';
              });
            },
            child: Text('Press Me'),
          ),
        ],
      ),
    ),
  );
 }
}



//corrected code

import 'package:flutter/material.dart';

void main() {
 runApp(MyApp());
}

class MyApp extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
  return MaterialApp(
    title: 'Flutter Demo',
```

```
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: HomeScreen(),
    );
  }
}

class HomeScreen extends StatefulWidget {
  @override
  _HomeScreenState createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  String _displayText=''; // Variable declared and initialized

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Home Screen'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            Text(_displayText), // Error: _displayText is used before being
initialized
            SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
                setState(() {
                  _displayText = 'Button Pressed';
                });
              },
```

```
            child: Text('Press Me'),
          ),
        ],
      ),
    ),
  );
 }
}
```

